
RESOURCE ALLOCATION IN DISTRIBUTED COMPUTING



Shangyi Shao

College of Computing and Data Science

A thesis submitted to the Nanyang Technological University
in partial fulfillment of the requirements for the degree of
Doctor of Philosophy

2026

Statement of Originality

I hereby certify that the work embodied in this thesis is the result of original research, is free of plagiarised materials, and has not been submitted for a higher degree to any other University or Institution.

Aug. 2018

• • • • •

Date

Signature

Signature

Shangyi Shao

Supervisor Declaration Statement

I have reviewed the content and presentation style of this thesis and declare it is free of plagiarism and of sufficient grammatical clarity to be examined. To the best of my knowledge, the research and writing are those of the candidate except as acknowledged in the Author Attribution Statement. I confirm that the investigations were conducted in accord with the ethics policies and integrity standards of Nanyang Technological University and that the research data are presented honestly and without prejudice.

Mar. 2019

.....

Date

NTU NTU NTU NTU NTU NTU NTU NTU
NTU NTU NTU NTU NTU NTU NTU NTU
Signature
NTU NTU NTU NTU NTU NTU NTU NTU
NTU NTU NTU NTU NTU NTU NTU NTU

Assoc. Prof. Xueyan Tang

Authorship Attribution Statement

Please select one of the following; *delete as appropriate:

*(A) This thesis does not contain any materials from papers published in peer-reviewed journals or from papers accepted at conferences in which I am listed as an author.

*(B) This thesis contains material from [x number] paper(s) published in the following peer-reviewed journal(s) / from papers accepted at conferences in which I am listed as an author.

Please amend the typical statements below to suit your circumstances if (B) is selected.

Chapter 4 is published as [D.T. Murphy, S. Schmid, J.R. Hester, P.E.R. Blanchard, and W. Müller. Coordination site disorder in spinel-type LiMnTiO₄. Inorganic Chemistry 54, 4636-4643 \(2015\). DOI: 10.1021/ic502747p.](#)

The contributions of the co-authors are as follows:

- A/Prof Schmid provided the initial project direction and edited the manuscript drafts.
- I prepared the manuscript drafts. The manuscript was revised by Dr Hester and Dr. Blanchard.
- I co-designed the study with A/Prof Siegbert Schmid and performed all the laboratory work at the School of Materials Science and Engineering and the Singapore Synchrotron Light Source. I also analyzed the data.
- All microscopy, including sample preparation, was conducted by me in the Facility for Analysis, Characterization, Testing and Simulation.
- Dr James Hester assisted in the collection of the neutron powder diffraction data.
- Dr Peter Blanchard assisted in the interpretation of the X-ray absorption spectroscopy data and carried out the spectral interpretation.
- Dr Wojciech Müller assisted in the collection and provide guidance in the interpretation of the magnetic measurement data.

Chapter 5 is published as [H. V Doan, B. Yao, Y. Fang, A. Sartbaeva, U. Hintermair, V. P Ting, Controlled Formation of Hierarchical Metal-Organic Frameworks using CO₂ Expanded Solvent Systems. In press, ACS Sustainable Chemistry & Engineering \(2017\). DOI: 10.1021/acssuschemeng.7b01429](#)

- Prof Ting suggested the materials area and edited the manuscript drafts.
- I wrote the drafts of the manuscript. The manuscript was revised together with Dr. Sartbaeva and Dr. Yao.
- II performed all the materials synthesis, collected X-ray diffraction patterns and visible light spectra, carried transmission electron microscopy, and conducted data evaluation.
- IDr. Y. Fang conducted the Rietveld analysis of the powder X-ray diffraction data and single crystal structure determinations.
- IDr U. Hintermair conducted the molecular dynamics simulations.
- IMs. A. Sartbaeva prepared the samples for electron microscopy.

.....

Date

Signature

Shangyi Shao

Acknowledgements

I wish to express my greatest gratitude to my advisor, Assoc. Prof Xueyan Tang. Without his full support, it would be impossible for me to

“If I had one hour to save the world, I would spend 55 minutes defining the problem and only five minutes finding the solution.”

—Einstein, Albert

To my dear family

Abstract

The proliferation of computing-intensive workloads has firmly established geo-distributed architectures—spanning cloud platforms, interconnected data centers, and high-performance computing clusters—as the standard paradigm for modern large-scale data processing. To exploit data locality and minimize bandwidth consumption, these systems routinely distribute computing jobs into multiple localized tasks deployed across disparate geographical sites where the requisite input data reside. This framework has become highly prevalent across diverse application domains, including sensor networks, climate modeling, and large-scale machine learning. However, the heterogeneous nature of these environments introduces profound challenges in resource allocation and job scheduling. Due to the imbalanced distribution of workloads and varying site capacities, classical single-cluster scheduling mechanisms often precipitate resource bottlenecks, system inefficiencies, and severe job starvation. Furthermore, while geo-distributed sites are typically interconnected via well-provisioned networks, the strategic utilization of these network resources to transfer data and computational demands across sites remains a complex, NP-hard problem. More critically, a fundamental dichotomy exists between optimizing system efficiency (e.g., minimizing average job response time) and guaranteeing equitable resource allocation among competing jobs. Purely efficiency-driven algorithms often compromise fairness, while strict fairness-oriented policies inevitably degrade overall throughput. To address these multi-dimensional challenges, this dissertation systematically investigates resource allocation and job scheduling in geo-distributed systems, proposing a series of novel, network-aware scheduling frameworks that leverage inter-site bandwidth to synergistically optimize processing efficiency and rigorous fairness guarantees.

The first phase of this research focuses on redefining equitable resource allocation in distributed execution environments by integrating network transmission capabilities. Existing studies on fair resource allocation, such as Max-min Fairness and Dominant Resource Fairness, predominantly operate under single-site assumptions

where all systemic resources are homogeneously accessible. In geo-distributed systems, however, evaluating fairness solely at the localized level is inherently flawed; a resource-intensive job constrained to a low-capacity site may suffer from highly unfair allocations despite monopolizing local resources. To resolve this, this dissertation proposes Dynamic Aggregate Max-min Fairness (DAMF), a pioneering resource allocation policy that leverages spare inter-site network bandwidth to dynamically transfer data and computation demands. DAMF mandates that the aggregate resource allocation for each job across all computational sites achieves max-min fairness over the entirety of the system’s demand transmission and allocation solution space. Theoretically, this research rigorously proves that the DAMF policy satisfies three critical axiomatic properties of fair division: Pareto efficiency, envy-freeness, and strategy-proofness. From a practical standpoint, the study develops comprehensive algorithms and a fast approximation tailored to compute optimal demand transmissions and precise DAMF resource allocations subject to dynamic demand distributions and strict network capacity constraints. Extensive experimental evaluations demonstrate that DAMF significantly outperforms conventional baseline policies, successfully achieving superior fairness metrics among competing jobs while simultaneously enhancing the aggregate utility of the distributed system.

Building upon the integration of network resources, the second phase of this dissertation pivots to the optimization of system efficiency, specifically targeting the minimization of average job response time in distributed environments. In geo-distributed architectures, a job’s completion time is strictly dictated by the completion of its terminal task across all executing sites. Consequently, traditional efficiency-centric strategies like Shortest Remaining Processing Time (SRPT), which are optimal for single-site clusters, yield deeply suboptimal results in distributed settings. The problem is exacerbated by the intricate interdependencies between the local execution order of jobs and the cross-site transmission of tasks: transferring a single task can fundamentally alter the global optimal execution sequence, while a newly proposed execution order may necessitate a divergent transmission strategy. To disentangle this complexity, this research introduces Workload-Aware Selection with Transmission (WAST), an integrated job scheduling algorithm designed to concurrently compute the global execution order and optimal inter-site

task transmissions. WAST systematically formulates the response-time minimization objective into a sequence of optimization problems, which are elegantly transformed into multiple minimum-cost flow problems solvable by advanced operational research solvers (e.g., OR-Tools, Cplex, GUROBI). To mitigate the substantial computational overhead inherent to the massive solution space, a highly efficient heuristic approximation algorithm is further proposed without significantly compromising scheduling fidelity. Driven by real-world production cluster traces from Alibaba, the empirical validation confirms that both WAST and its heuristic counterpart compute near-optimal execution sequences and transmission strategies, drastically reducing average job response times, particularly under highly skewed spatial workload distributions.

The final phase of the dissertation unifies the dual objectives by establishing a robust framework that successfully reconciles the inherent trade-off between scheduling efficiency and long-term fairness. Recognizing that instantaneous fairness can cripple system throughput and pure efficiency can induce prolonged job starvation, this research introduces a stringent long-term fairness criterion inspired by the concept of "protective scheduling." Under this criterion, a scheduling policy must guarantee that no job experiences a completion time later than it would under a baseline distributed Processor Sharing (PS) protocol, which theoretically guarantees equitable capacity sharing among all active jobs. To operationalize this, the dissertation formulates the distributed protective scheduling problem by constructing a mathematically rigorous slack system characterized by dual perspectives: a global view and an independent view. Crucially, mathematical proofs presented in this study reveal that, owing to the volatile nature of dynamic job arrivals and shifting load distributions, no purely globally order-based scheduling policy can maintain strict protectiveness. To transcend this theoretical limitation, a novel heuristic algorithmic framework, denoted as PFSP_x, is developed. PFSP_x acts as a hybrid scheduling engine that strictly enforces the protective fairness boundaries while aggressively optimizing the internal processing sequences to maximize efficiency. Trace-driven comparative analyses unequivocally demonstrate that PFSP_x effectively eliminates the violations observed in globally order-based extensions and highly aggressive efficiency algorithms (such as SRPT and SWAG). Ultimately, PFSP_x achieves the highest job processing efficiency among all rigorously protective policies, providing a low-cost, resilient scheduling paradigm for modern, large-scale geo-distributed computing systems.

Contents

List of Figures

List of Tables

Symbols and Acronyms

Symbols

$\mathcal{R}^n$	the n -dimensional Euclidean space
$\mathcal{H}$	the Euclidean space
$\ \cdot\ $	the 2-norm of a vector or matrix in Euclidean space
$\ \cdot\ _G$	the induced norm of a vector in G-space
$\ \cdot\ _E$	the induced norm of a vector or matrix in probabilistic space
$\odot$	the Hadamard (component-wise) product
$\otimes$	the Kronecker product
$\langle \cdot, \cdot \rangle$	the inner product of two vectors
$\circ$	the composition of functions
∇f	the gradient vector
$\mathcal{C}^k$	the function with continuous partial derivatives up to k orders
$x_{i,k}$	the i -th component of a vector x at time k
$\bar{x}$	the vector with the average of all components of x as each element
$\mathbf{1}$	all-ones column vector with proper dimension
$\mathcal{C}$	the average space, i.e., $\text{span}\{\mathbf{1}\}$
$\mathcal{C}^\perp$	the disagreement space, i.e., $\text{span}^\perp\{\mathbf{1}\}$
$\Pi_\parallel$	the projection matrix to the average space $\mathcal{C}$
$\Pi_\perp$	the projection matrix to the disagreement space $\mathcal{C}^\perp$
$O(\cdot)$	order of magnitude or ergodic convergence rate (running average)
$o(\cdot)$	non-ergodic convergence rate
$\mathcal{N}_i$	the index set of the neighbors of agent i

Acronyms

DOP	Distributed Optimization Problem
EDOP	Equivalent Distributed Optimization Problem
SDOP	Stochastic Distributed Optimization Problem
OEP	Optimal Exchange Problem
OCF	Optimal Consensus Problem
DOCP	Dynamic Optimal Consensus Problem
AugDGM	Augmented Distributed Gradient Methods
AsynDGM	Asynchronous Distributed Gradient Methods
D-ESC	Distributed Extremum Seeking Control
D-SPA	Distributed Simultaneous Perturbation Approach
D-FBBS	Distributed Forward-Backward Bregman Splitting
ADMM	Alternating Direction Method of Multipliers
DSM	Distributed (Sub)gradient Method
GAS	Globally Asymptotically Stable
UGAS	Uniformly Globally Asymptotically Stable
SPAS	Semi-globally Practically Asymptotically Stable
USPAS	Uniformly Semi-globally Practically Asymptotically Stable
HoS	Heterogeneity of Stepsize
FPR	Fixed Point Residual
OBE	Objective Error
i.i.d.	independent and identically distributed
<i>a.s.</i>	almost sure convergence of a random sequence

Chapter 1

Introduction

1.1 Research Scope

It is increasingly common for data-processing jobs in geo-distributed systems to spread tasks among multiple sites for data locality: each task will be deployed and processed at some site where their input data are available [? ? ?]. Through this framework, the system can effectively save the bandwidth cost and improve the service response time. Geo-distributed processing has been prevalent for data analytics in various domains such as sensor networks [?], climate science [? ?] and stock exchange [?]. In such systems, providing fair resource allocation to jobs can help with preventing starvation and fulfilling QoS for all jobs, but it is a non-trivial problem. The problem consists of two critical hurdles: how to decide the distributed resource allocation for each job with various amount of resource demand distribution across sites, and how to utilize the limited network resources to transfer tasks of certain jobs between sites to achieve more fair allocation and better overall utility.

Existing work on fair resource allocation has primarily focused on a cluster of machines at a single site, where all resources in the system are assumed to be available for processing all jobs. Typical examples include the classical Max-min Fairness [?] and Proportional Fairness [?]. As an extension, Constrained Max-min Fairness [?] achieves max-min fair allocation given the placement constraints of jobs on the machines. Dominant Resource Fairness [?] and Resource Elasticity Fairness [?] present fair allocations of multiple resources. On the other hand, the study of

distributed job execution has mainly focused on improving job scheduling efficiency or saving bandwidth cost [? ?]. Little work has considered fair resource allocation among jobs executing over multiple sites. Recently, Guan et al. [?] has explored the conventional max-min fairness in distributed job execution by requiring the aggregate resource allocation across all sites to be max-min fair. However, if the job workloads are distributed unevenly among the sites, the resulting resource allocation could be quite inefficient or unfair with respect to each job's demand. For example, if a high-demand job's entire workload is located at one site with small capacity, its allocation can still be unfair even if it takes up all resources at that site.

As different sites in the distributed system are often inter-connected with well-provisioned networks, the spare bandwidth can be utilized to transfer data and hence computation demands among sites to enhance the efficiency and improve the fairness of resource allocation among jobs. Since there is a huge solution space, the choice of moving which job's data from which site to another is a challenge. To the best of our knowledge, achieving fair resource allocation with exploiting the use of idle network resources has not been widely studied. In this paper, we apply max-min fairness to distributed job execution with utilization of network resources to transfer job demands between sites. We propose a resource allocation policy called Dynamic Aggregate Max-min Fairness (DAMF), which requires each job's aggregate resource allocation across all sites to be max-min fair among all possible demand transmissions and resource allocations. We prove that DAMF satisfies three important properties of fairness: Pareto efficiency, envy-freeness and strategy-proofness. Given the demand distributions of jobs and the network capacity constraints, we develop algorithms to compute the demand transmission and the DAMF resource allocation to each job at each site. Experimental results demonstrate that compared with existing allocation policies, DAMF can achieve resource allocations with better fairness among jobs and improve the system utility.

Meanwhile, this kind of geo-distributed computing systems introduces many new challenges, such as how to improve system efficiency and decrease the volume of data transmission. In a single data center scenario, the strategy for achieving the highest job processing efficiency (i.e., minimize the average job response time – the response time of a job is defined as the duration between its arrival time and its completion time) is to sort and execute jobs based on their computation loads

(i.e., the total processing time of all tasks of each job) from low to high, known as the Shortest Remaining Processing Time (SRPT) strategy. Similar ideas can also be applied to the distributed job execution scenario where jobs are comprised of tasks to be processed at multiple data centers, but this can be far from optimal. In geo-distributed job execution, the completion time of each job is determined by the completion time of its last task across all sites. Therefore, the execution order of jobs within individual sites leaves a huge space for optimization, especially when the task distributions of jobs among different sites are imbalanced. How to achieve optimal job scheduling in distributed job execution is an NP-hard problem.

What makes the problem even more complicated is how to utilize network resources in the system to further enhance job processing efficiency. Since the completion time of the last task decides the response time of a job, transferring the input data of tasks to other sites can potentially bring forward the execution of tasks and thereby improve the job's response time. However, there exists complex interdependency between the execution order of jobs and the transmission of tasks. Even the transmission of a single task may change the optimal execution order of jobs, while different job execution orders may, in turn, require different task transmissions to minimize the average job response time. As a result, it is highly challenging to find the optimal solution.

In this paper, we focus on developing job scheduling algorithms which utilize network resources to transfer tasks and improve job processing efficiency in geo-distributed systems. We present a novel job scheduling strategy, called Workload-Aware Selection with Transmission (WAST), designed to optimize average job response time. WAST computes the global execution order of jobs and a set of task transmissions between different sites in an integrated manner. More specifically, WAST searches for the task transmission strategy to achieve the best possible response time of each job by formulating an optimization problem and transforming it into multiple minimum-cost flow problems that can be efficiently solved by popular solvers (e.g., OR-Tools [?], Cplex [?], and GUROBI [?]), and iteratively schedules the job with the minimum response time until all jobs are scheduled. We also present a heuristic approximation algorithm that reduces the computational complexity without compromising the quality of scheduling much. We validate the effectiveness of the WAST algorithm and its approximation using publicly available

job traces from Alibaba. Experimental results show that both algorithms can efficiently compute near-optimal job execution order and task transmissions, thereby significantly reducing job response times, especially in scenarios where the initial computation load distributions of jobs are highly skewed across sites.

1.2 Thesis Contributions

Chapter ?? introduces ...

Chapter ?? reviews ...

More chapters

....

Chapter 2

Literature Review

2.1 Fairness in Resource Allocation in Single-Site Systems

Max-Min Fairness (MMF) is a widely used resource allocation policy. Radunovic *et al.* Ghodsi *et al.* [?] proposed Dominant Resource Fairness (DRF) which decides a user's allocation by its dominant share, i.e., the maximum proportional share that the user is allocated among all resource types. Ghodsi *et al.* [?] addressed task placement constraints and presented Constrained Max-min Fairness, which tries to meet the fairness target by recursively satisfying the users based on the order of dissatisfaction. Zahedi and Lee [?] solved the multi-resource allocation problem by proportionally allocating different resources based on the re-scaled Cobb-Douglas Utility, and named it Resource Elasticity Fairness. Gutman and Nisan [?] further extended DRF with respect to the bundle measure and utility functions. Multi-resource allocation holds applications in cloud servers [?], heterogeneous GPU clusters [?], and 5G networks [?]. Our work also considers multiple types of resources, but we focus on utilizing network resources to enhance the fairness of compute resource allocation.

In geo-distributed systems, a job may have tasks spread over different sites. Hung *et al.* [?] presented a scheduling algorithm for distributed job execution, which selects the job with the least overall demand to run first in order to minimize the average response time of jobs. This allocation policy is straightforward but

apparently unfair, and could cause starvation for large-demand jobs. Huang *et al.* [?] and Chen *et al.* [?] studied the job scheduling problem in distributed cloud services and tried to achieve max-min fairness for jobs in terms of the job response time. Different from the above work, we aim to achieve max-min fairness in the instantaneous resource allocation among jobs across geo-distributed sites given the availability of network resources. Recent studies have also adopted deep reinforcement learning to search for effective resource allocation strategies in fields like wireless networks [?], edge computing [?] and satellite networks [?]. However, learning-based approaches have mainly focused on improving the efficiency of resource allocation (such as system utility, revenue and profit) rather than achieving the fairness of resource allocation.

2.2 Fairness in Resource Allocation in Distributed Job Execution

The task (job) scheduling problem in geo-distributed environments has been widely studied in many aspects. Some research focuses on utilizing the game theory to minimize the system entity cost by assuming that users in the system have different interests, and thus they form a game model [?]. Many efforts aim to minimize the bandwidth usage or the transmission time/cost in the system. For example, [?] focuses on optimizing the transmission time of data along the given paths, Gaia [?] tries to minimize the transmission cost between machine learning clusters while maintaining the model's performance, and ELIS [?] aims to save the network bandwidth usage on both intra-DC and inter-DC links while ensuring the QoS of microservices. Previous studies have also made progress on developing efficient distributed job scheduling algorithms. However, they typically rely on simplified system assumptions which limit their applicability in real practice [?]. For example, Hung *et al.* [?] presents a workload-aware strategy which schedules jobs based on their estimated completion times under given task placements, but it neglects to utilize network resources to move tasks. Guan *et al.* [?], Zhao *et al.* [?] study the task assignment problem given that the input data of each task can be replicated across multiple geo-distributed data centers, and they do not consider using network resources either. Hu *et al.* [?] and Pu *et al.* [?] introduce algorithms considering task transmissions, but they focus on improving

the efficiency of a single job and do not study the scheduling strategy across jobs. Chen *et al.* [?] presents a scheduling algorithm that incorporates transmissions but imposes strict constraints that the number of tasks is no more than the number of computing slots in the system. Convolbo *et al.* [?] coordinates scheduling with transmission but focuses on minimizing the makespan of job execution instead of the response times of individual jobs.

2.3 Efficient Scheduling in Distributed Job Execution

Job scheduling and resource allocation are fundamental problems in system processing, which have been widely studied from a variety of system scenarios and performance objectives [? ? ? ?]. In a single-site scenario, the Shortest Remaining Processing Time (SRPT) policy is universally recognized as the optimal policy for maximizing job processing efficiency, whereas a multitude of notions exist for different types of fair resource allocation. Radunovic *et al.* [?] proposed a unified framework for max-min fairness, which aims to maximize the utility of the least satisfied users. In addition, Ghodsi *et al.* [?] proposed Dominant Resource Fairness (DRF) under the multiple resource scenario, which decides a user's allocation by its dominant share, i.e., the maximum proportional share that the user is allocated among all resource types. Zahedi and Lee [?] solved the multi-resource allocation problem based on the re-scaled Cobb-Douglas Utility, and proportionally allocates different resources to each user. Fossati *et al.* [?] presents a distributed slice multi-resource allocation algorithm specialized for 5G systems. In particular, Friedman *et al.* [? ?] presents the notion of protective scheduling, which sets the criterion of fairness to be that no job would finish later than its completion time under the PS policy. Under this fairness guarantee, they present a series of algorithm to further improve the job processing efficiency.

In distributed job execution case, since the job load and the system capacity are distributed (mostly unevenly) across multiple sites, the problem becomes more complex due to the additional dimension of site-specific resource constraints and inter-site communication overheads. Although the SRPT policy can be extended to such distributed cases, it admits multiple algorithmic variations and they all

no longer guarantees optimality. Hung *et al.* [?] proposed a workload-aware heuristic scheduling algorithm namely SWAG, which iteratively schedules the job with the earliest completion time while taken the scheduled workload in the system into consideration, and their empirical evaluations demonstrated that their proposed algorithm achieves superior performance compared to SRPT extensions. In addition, Guan *et al.* [?] and Zhao *et al.* [?] proposed algorithms to optimize the job completion time in geo-distributed systems under the case that the required data trunk for each task is replicated at multiple sites. Convolbo *et al.* [?] studied the utilization of network resources, while their objective is to minimize the system makespan within geo-distributed systems, i.e., the total time required to process all jobs. In the context of fairness, Guan *et al.* [?] introduced a task assignment algorithm aimed at achieving max-min fairness in aggregate resource allocation among jobs in the distributed job execution. Similarly, Chen *et al.* [?] investigated achieving max-min fairness in terms of the job response times in geo-distributed systems by task transmission. Different from the works above, we focus on the dual objective of efficiency and long-term fairness in geo-distributed job execution without transmission.

Chapter 3

Achieving Max-min Fairness in Distributed Job Execution with Task Transmissions

3.1 System Model

Consider a geo-distributed system composed of m separate sites: $S_1, S_2, \dots, S_m$. Each site S_j holds compute resources with a capacity u_j . The capacity can be interpreted as the volume of computing slots that are available.

We would like to conduct fair resource allocations among jobs in an online environment where jobs arrive dynamically for execution. To do so, we start by studying a static problem of defining and deriving a fair instantaneous resource allocation among active jobs at a certain snapshot of the system. Suppose n jobs $J_1, J_2, \dots, J_n$ are running in the system. Each job J_i consists of tasks that process distinct data partitions and could run in parallel. Due to the data locality consideration, initially each task is restricted to be served at a certain site where the input data is available. Therefore, a job's resource demand on a site S_j can be defined as the total demand of its tasks that can only run on S_j . For instance, if each task needs one computing slot for execution, the resource demand on site S_j is given by the number of tasks to run on S_j . We model the resource demands of all jobs as an $n * m$ demand matrix D_{n*m} , where each element d_{ij} denotes job J_i 's demand on site S_j . Similarly, the instantaneous resource allocations to all jobs on all sites can

be represented by an $n * m$ allocation matrix A_{n*m} , where each element a_{ij} denotes how much resource job J_i is allocated on site S_j . Throughout this paper, we shall use capital letters to stand for matrices and corresponding small letters to stand for the elements therein. For simplicity, we use $[x]$ to represent the set $\{1, 2, \dots, x\}$ for a positive integer x .

Some basic constraints should be met to ensure that A_{n*m} is a sensible allocation. First, the summation of allocated resources on any site should not exceed that site's capacity:

$$\forall j \in [m], \quad \sum_{i=1}^n a_{ij} \leq u_j. \quad (3.1)$$

Allocations satisfying (3.1) are said *feasible*.

Second, any job's allocation on any site should not exceed its demand on that site so that no resources are wasted:

$$\forall i \in [n], j \in [m], \quad 0 \leq a_{ij} \leq d_{ij}. \quad (3.2)$$

Allocations satisfying (3.2) are said *rational*.

Suppose there are networks connecting the sites and they can be used to transfer data and hence demand for compute resources from one site to another. We use an $n * m$ transmission matrix ΔD_{n*m} to represent the demand transmissions. Each element Δd_{ij} can be positive or negative: A positive Δd_{ij} describes how much demand of job J_i is transferred into site S_j , while a negative Δd_{ij} means some demand is transferred out of (removed from) site S_j , and $\Delta d_{ij} = 0$ represents no transmission. Apparently, we cannot transfer more demand out of a site than what a given job has at that site:

$$\forall i \in [n], j \in [m], \quad d_{ij} + \Delta d_{ij} \geq 0, \quad (3.3)$$

and the overall demand of each job J_i should be kept unchanged after the transmissions:

$$\forall i \in [n], \quad \sum_{j=1}^m \Delta d_{ij} = 0. \quad (3.4)$$

We assume that the primary bandwidth bottleneck of the system lies in the link through which each site connects to the backbone network. Thus, to model the available network capacity, we use a count limit c_j for each site S_j to describe an

upper bound on the total demand that can be transferred into and out of site S_j within a predefined short period of time T :

$$\forall j \in [m], \quad \sum_{i=1}^n |\Delta d_{ij}| \leq c_j. \quad (3.5)$$

In big data processing frameworks, each task typically processes a data partition/split of the same size [?]. The transmission count limit c_j can be computed based on the data partition size, the amount of bandwidth currently available, and T .

Upon the transmissions, each job J_i 's demand on site S_j becomes $d_{ij} + \Delta d_{ij}$, i.e., the demand matrix becomes $D + \Delta D$. Hence, constraint (??) is rewritten as:

$$\forall i \in [n], j \in [m], \quad 0 \leq a_{ij} \leq d_{ij} + \Delta d_{ij}. \quad (3.6)$$

We shall start with defining and deriving a fair resource allocation $A_{n \times m}$ among jobs satisfying constraints (??), (??), (??), (??) and (??), given the demands $D_{n \times m}$, site capacities u_j 's and transfer limits c_j 's. Later, we shall discuss how to apply the solution to this static problem to online scenarios where jobs arrive dynamically. The key notations used in this paper are summarized in Table ??.

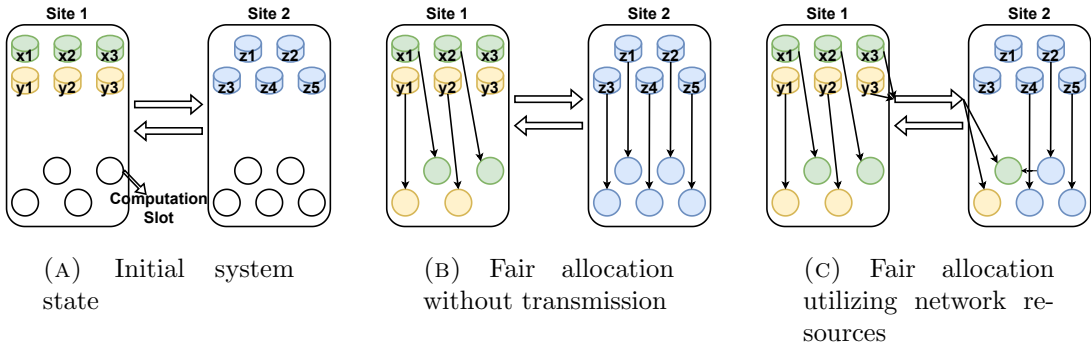


FIGURE 3.1: An intuitive example

Figure ?? provides an example of the static problem to illustrate that transferring job demands can help improve the fairness of resource allocation. Suppose a geo-distributed system has 2 sites S_1 and S_2 with capacities of 4 and 5 computing slots respectively. There are three active jobs x , y and z in the system. As shown in Figure ??, jobs x and y have 3 tasks each, where the input data is available on site S_1 ; job z has 5 tasks, where the input data is available on site S_2 . If no demand

can be transferred between sites, the most balanced resource allocation among jobs x , y and z is 2, 2 and 5 computing slots as illustrated in Figure ??, because jobs x and y do not have demands on site S_2 . With a transfer limit of 2, we can transfer one task of job x and one task of job y from S_1 to S_2 , and allocate them with computing slots on S_2 . As a result, all jobs x , y and z are allocated 3 computing slots each as shown in Figure ??, which is more balanced (fair) from the global perspective.

TABLE 3.1: Notation Table

Notation	Definition
$\mathcal{M} = \{S_1, S_2, \dots, S_m\}$	Set of sites
$\mathcal{J} = \{J_1, J_2, \dots, J_n\}$	Set of jobs
u_j	Site S_j 's compute resource capacity
c_j	Site S_j 's transmission count limit
$D_{n \times m}$	Demand matrix
$\Delta D_{n \times m}$	Transmission matrix
$A_{n \times m}$	Allocation matrix
$\mathcal{X}$	Set of all allocation matrices
X	Set of all aggregate allocation vectors

3.2 Preliminaries

3.2.1 Max-min Fairness

Max-Min Fairness (MMF) is one of the most common resource allocation strategies, which takes “serving the most unsatisfied user” as priority. Considering the resource allocation among n jobs as an n -dimensional vector, MMF can be defined as follows [?]:

Definition 3.1. Consider a set V of n -dimensional vectors. A vector $\vec{x} \in V$ is max-min fair on set V , if and only if for any $\vec{y} \in V$, if $y_s > x_s$ for an index $s \in \{1, 2, \dots, n\}$, there must exist another index $t \in \{1, 2, \dots, n\}$ such that $y_t < x_t \leq x_s$.

Based on the above definition, if we want to increase a component x_s in a max-min fair vector $\vec{x}$ while keeping the new vector staying in the vector set V , we must decrease another component x_t in $\vec{x}$ whose original value is less than or equal to that

of x_s . The max-min fair vector is unique if it exists, because if there are multiple max-min fair vectors within a given vector set V , we can derive contradiction simply by applying the definition of MMF to them. Clearly, the max-min fair vector does not exist in every vector set. Previous work has proved sufficient conditions for the existence of the max-min fair vector [?].

Theorem 3.1. *If a set V is compact and convex, then it is max-min achievable.*

3.2.2 Aggregate Max-min Fairness

A naive idea of applying MMF to distributed job execution is to consider each site independently and find the MMF resource allocation among jobs on each site based on their local demands. This policy is known as Independent Max-min Fairness (IMF) [?]. However, IMF can be far away from providing fair services to jobs from the global perspective due to the possibly unbalanced distribution of job demands and compute capacities among sites.

Guan *et al.* [?] defined Aggregate Max-min Fairness (AMF) for distributed job execution and presented an algorithm to achieve AMF. AMF treats the distributed system as a united resource pool and constructs an aggregate allocation vector to represent each job's total resource allocation across all distributed sites:

$$\vec{A} = \langle \sum_{j=1}^m a_{1j}, \sum_{j=1}^m a_{2j}, \dots, \sum_{j=1}^m a_{nj} \rangle.$$

AMF requires this vector to be max-min fair among all possible aggregation vectors. While the AMF vector is unique if it exists, there may be multiple ways of resource allocation by sites to jobs producing this AMF vector since the vector only specifies each job's aggregate allocation across all sites.

AMF does not consider utilizing network resources to transfer job demands between sites. Transferring job demands has the potential to improve the fairness of compute resource allocation for jobs as well as the system utility.

3.3 Dynamic Aggregate Max-min Fairness

We propose a new resource allocation policy called Dynamic Aggregate Max-min Fairness (DAMF) to improve the fairness of resource allocation by making use of network resources. Given the demand matrix, site capacities and transfer limits, there are a variety of possible transmission matrices and each would give rise to a number of feasible and rational allocations. Under this circumstance, DAMF requires the aggregate allocation vector to be max-min fair among all the vectors derived from the allocation matrices resulting from all possible transmission matrices. Let $\mathcal{X}$ be the set of all feasible and rational allocation matrices under all possible transmission matrices, and X be the set of all aggregate allocation vectors drawn from $\mathcal{X}$. DAMF can be formally defined as follows:

Definition 3.2. An allocation $A_{n \times m}$ satisfies Dynamic Aggregate Max-min Fairness, if and only if its aggregate allocation vector $\vec{A}$ is max-min fair over the set X .

We prove that DAMF exists by showing the compactness and convexity of set X .

Lemma 3.2. *The set X is compact.*

Proof. Consider any aggregate allocation vector $\vec{A} = \langle \sum_{j=1}^m a_{1j}, \sum_{j=1}^m a_{2j}, \dots, \sum_{j=1}^m a_{nj} \rangle$ derived from some allocation matrix A under some demand matrix $D + \Delta D$ after transmission. According to constraints (??), (??) and (??), for the i -th component $\sum_{j=1}^m a_{ij}$, we have $0 \leq \sum_{j=1}^m a_{ij} \leq \sum_{j=1}^m \min\{u_j, d_{ij} + \Delta d_{ij}\} \leq \sum_{j=1}^m \min\{u_j, d_{ij} + c_j\}$. Thus, the set X is closed and bounded. Hence, it is compact. $\square$

Lemma 3.3. *The set X is convex.*

Proof. Based on the definition of convexity, we need to prove that for any two vectors $\vec{A}, \vec{B} \in X$ and any $0 \leq \lambda \leq 1$, it holds that $\lambda \vec{A} + (1 - \lambda) \vec{B} \in X$.

Let $\mathcal{T}$ be the set of all possible transmission matrices satisfying (??), (??) and (??). We first prove that $\mathcal{T}$ is convex. i.e., for any two transmission matrices $\Delta D, \Delta D' \in \mathcal{T}$ and any $0 \leq \lambda \leq 1$, $\lambda \cdot \Delta D + (1 - \lambda) \cdot \Delta D' \in \mathcal{T}$.

Firstly, $\lambda \cdot \Delta D + (1 - \lambda) \cdot \Delta D'$ satisfies constraint (??): since $\Delta D, \Delta D' \in \mathcal{T}$, for each job J_i and each site S_j , we have $d_{ij} + \Delta d_{ij} \geq 0$ and $d_{ij} + \Delta d'_{ij} \geq 0$, and hence,

$$\begin{aligned} & d_{ij} + \lambda \cdot \Delta d_{ij} + (1 - \lambda) \cdot \Delta d'_{ij} \\ &= \lambda \cdot (d_{ij} + \Delta d_{ij}) + (1 - \lambda) \cdot (d_{ij} + \Delta d'_{ij}) \\ &\geq \lambda \cdot 0 + (1 - \lambda) \cdot 0 = 0. \end{aligned}$$

Similarly, it can be shown that $\lambda \cdot \Delta D + (1 - \lambda) \cdot \Delta D'$ satisfies constraints (??) and (??). As a result, $\lambda \cdot \Delta D + (1 - \lambda) \cdot \Delta D' \in \mathcal{T}$ and thus, $\mathcal{T}$ is a convex set.

Next, suppose the original demand matrix is D , and vector $\vec{A}$ and $\vec{B}$'s corresponding allocation and transmission matrices are $\{A, \Delta D^A\}$ and $\{B, \Delta D^B\}$, respectively. Since the set $\mathcal{T}$ is convex, we have $\lambda \cdot \Delta D^A + (1 - \lambda) \cdot \Delta D^B \in \mathcal{T}$. By definition, $\lambda \vec{A} + (1 - \lambda) \vec{B}$ is the aggregate allocation vector for the allocation matrix $\lambda A + (1 - \lambda) B$. We prove that $\lambda A + (1 - \lambda) B$ is a rational and feasible allocation matrix for the demand matrix $D + \lambda \cdot \Delta D^A + (1 - \lambda) \cdot \Delta D^B$.

Since A and B are feasible, by constraint (??) we have

$$\forall j \in [m], \quad \sum_{i=1}^n a_{ij} \leq u_j, \quad \sum_{i=1}^n b_{ij} \leq u_j.$$

Thus,

$$\sum_{i=1}^n (\lambda a_{ij} + (1 - \lambda) b_{ij}) \leq \lambda u_j + (1 - \lambda) u_j = u_j.$$

Hence, $\lambda A + (1 - \lambda) B$ is feasible. Similarly, it can be shown that $\lambda A + (1 - \lambda) B$ is rational. Therefore, $\lambda A + (1 - \lambda) B \in \mathcal{X}$. As a result, $\lambda \vec{A} + (1 - \lambda) \vec{B} \in X$, and X is a convex set. □

Lemmas ?? and ?? together with Theorem ?? give rise to the following conclusion:

Theorem 3.4. *Dynamic Aggregate Max-min Fairness is achievable.*

3.4 Properties of DAMF

In this section, we study whether DAMF satisfies the properties of Pareto efficiency, envy-freeness and strategy-proofness, which are well-accepted characteristics of fairness.

3.4.1 Pareto Efficiency

Pareto efficiency means that no further improvement can be achieved which makes at least one individual better off without making any other individual worse off [?]. This includes two aspects [?]: first, no resource required by any consumer is left idle; second, no consumer is allocated any resource that it does not need.

In our distributed job execution context, suppose the DAMF vector and its corresponding allocation matrix are $\vec{A}$ and $A_{n \times m}$ respectively. Pareto efficiency means that for any other feasible and rational allocation matrix $A'_{n \times m}$ and its corresponding aggregate allocation vector $\vec{A}'$, if there is an index i such that the i -th component of $\vec{A}$ is larger than the i -th component of $\vec{A}'$, i.e., $\sum_{j=1}^m a'_{ij} > \sum_{j=1}^m a_{ij}$, there must be another index k such that the k -th component of $\vec{A}$ is smaller than the k -th component of $\vec{A}'$, i.e., $\sum_{j=1}^m a'_{kj} < \sum_{j=1}^m a_{kj}$.

Theorem 3.5. *DAMF is Pareto efficient.*

Proof. By definition, the DAMF vector is the max-min fair vector among all possible aggregate allocation vectors under all possible transmission matrices. For any other feasible and rational allocation matrix $A'_{n \times m}$ and its corresponding aggregate allocation vector $\vec{A}'$, if $\sum_{j=1}^m a'_{ij} > \sum_{j=1}^m a_{ij}$ for some job J_i , then based on Definition ??, there exists another job J_k such that $\sum_{j=1}^m a'_{kj} < \sum_{j=1}^m a_{kj} \leq \sum_{j=1}^m a_{ij}$. Hence, the theorem is proven. $\square$

3.4.2 Envy-freeness

Envy-freeness means that no individual would envy another individual's allocation. More specifically, in our distributed job execution context, envy-freeness means that for any job J_l , its usable allocation will not increase if it is given any other job J_k 's transmission and allocation.

Theorem 3.6. *DAMF is envy-free.*

Proof. Let $\vec{A}$ be the DAMF vector, and $A_{n \times m}$, $\Delta D_{n \times m}$ be its corresponding allocation and transmission matrices respectively. Assume on the contrary that they are not envy-free, that is, a job J_l will have its usable allocation increased if it is given another job J_k 's transmission Δd_{kj} 's and allocation a_{kj} 's. Note that the transmission refers to the network bandwidth allocated, and J_l does not have to fully utilize the allocated bandwidth in order to optimize its usable allocation. Let r_{lj} 's be the actual transmission used by J_l , where for each site S_j , $|r_{lj}| \leq |\Delta d_{kj}|$ (transmission is within the allocated bandwidth), $d_{lj} + r_{lj} \geq 0$ (transmission is feasible), and $\sum_{j=1}^m r_{lj} = 0$ (the overall demand of J_l is unchanged after transmission). Then, the demands of J_l on all sites would become $d_{lj} + r_{lj}$'s. Thus, J_l 's usable allocation on each site S_j is $\min\{d_{lj} + r_{lj}, a_{kj}\}$. If J_l 's total usable allocation over all sites increases, it means that

$$\sum_{j=1}^m \min\{d_{lj} + r_{lj}, a_{kj}\} > \sum_{j=1}^m a_{lj}.$$

This implies two facts:

(i) By eliminating the min operator, we have

$$\sum_{j=1}^m a_{kj} > \sum_{j=1}^m a_{lj},$$

which means job J_k 's aggregate allocation is larger than J_l 's aggregate allocation in the DAMF vector.

(ii) There must be at least one site $S_{j'}$ such that $\min\{d_{lj'} + r_{lj'}, a_{kj'}\} > a_{lj'}$, which indicates $d_{lj'} + r_{lj'} > a_{lj'}$ and $a_{kj'} > a_{lj'}$. The former inequality means that J_l 's demand on site $S_{j'}$ is not completely satisfied, and the latter inequality means that J_k 's allocation on site $S_{j'}$ is larger than J_l 's allocation on $S_{j'}$.

By (ii), from the DAMF allocation matrix $A_{n \times m}$, we could increase $a_{lj'}$ by decreasing $a_{kj'}$ on site $S_{j'}$. Accordingly, J_l 's aggregate allocation increases while J_k 's aggregate allocation decreases. Note that by (i), J_k 's aggregate allocation is larger than J_l 's aggregate allocation in $A_{n \times m}$. This implies that we can increase the l -th component of the DAMF vector $\vec{A}$ by decreasing its k -th component which is larger, and this

contradicts the fact that $\vec{A}$ is the max-min fair vector among all possible aggregate allocation vectors. $\square$

3.4.3 Strategy-proofness

Strategy-proofness means that no individual could achieve better allocation by lying about its demand. In our distributed job execution context, strategy-proofness requires that no job could get higher usable allocation by lying about its demand d_{ij} 's, even if this would cause the recomputation of the transmission matrix ΔD and the allocation matrix A .

Theorem 3.7. *DAMF is strategy-proof.*

Proof. Let $\vec{A}$ be the DAMF vector, and $A_{n \times m}$, $\Delta D_{n \times m}$ be its corresponding allocation and transmission matrices derived from the true demand matrix $D_{n \times m}$ (i.e., when all jobs are honest). Let $D'_{n \times m}$ be the misclaimed demand matrix when a job J_l lies about its demand, and $A'_{n \times m}$ and $\Delta D'_{n \times m}$ be the resultant allocation matrix and transmission matrix respectively. Note that the transmission refers to the network bandwidth allocated. J_l does not have to fully utilize the allocated bandwidth in order to optimize its usable allocation. Let r_{lj} 's be the actual transmission used by J_l , where for each site S_j , $|r_{lj}| \leq |\Delta d'_{lj}|$ (transmission is within the allocated bandwidth), $d_{lj} + r_{lj} \geq 0$ (transmission is feasible for J_l 's true demand), and $\sum_{j=1}^m r_{lj} = 0$ (the overall demand of J_l is unchanged after transmission). For notational convenience, we define R as the transmission matrix actually used by all jobs, i.e., R replaces $\Delta d'_{lj}$'s with r_{lj} 's for job J_l and keeps all other elements unchanged in $\Delta D'$. Then, the real demand matrix after transmission is given by $D + R$. In particular, the real demands of J_l at all sites are $d_{lj} + r_{lj}$'s. Thus, J_l 's usable allocation on each site S_j is $\min\{d_{lj} + r_{lj}, a'_{lj}\}$. We prove that J_l 's total usable allocation over all sites cannot increase, i.e.,

$$\sum_{j=1}^m \min\{d_{lj} + r_{lj}, a'_{lj}\} \leq \sum_{j=1}^m a_{lj}.$$

We prove it by contradiction. Assume on the contrary that J_l 's total usable allocation over all sites increases:

$$\sum_{j=1}^m \min\{d_{lj} + r_{lj}, a'_{lj}\} > \sum_{j=1}^m a_{lj}. \quad (3.7)$$

It follows that

$$\sum_{j=1}^m a'_{lj} > \sum_{j=1}^m a_{lj}. \quad (3.8)$$

Since J_l lies, it is possible that it gets excessive allocation than its real demand on some sites. Next, we construct a rational allocation matrix B' by shrinking J_l 's excessive allocation (if any) in A' to its real demand:

(i) For each job $J_i \neq J_l$, let $b'_{ij} = a'_{ij}$. Since these jobs are honest about their demands, their allocations must be rational:

$$\forall j \in [m], \quad b'_{ij} = a'_{ij} \leq d_{ij} + \Delta d'_{ij} = d_{ij} + r_{ij}.$$

(ii) For job J_l , if its allocation a'_{lj} on site S_j does not exceed its real demand $d_{lj} + r_{lj}$, it is kept the same in B' :

$$\forall j \in [m] \text{ where } a'_{lj} \leq d_{lj} + r_{lj}, \quad b'_{lj} = a'_{lj}.$$

(iii) For job J_l , if its allocation a'_{lj} on site S_j exceeds its real demand $d_{lj} + r_{lj}$, we shrink it to be the real demand in B' :

$$\forall j \in [m] \text{ where } a'_{lj} > d_{lj} + r_{lj}, \quad b'_{lj} = d_{lj} + r_{lj}.$$

In short, B' records the allocations in A' that can be actually used by all jobs. It is clear that B' is a feasible and rational allocation matrix given the real demand matrix $D + R$. For simplicity, B' can be uniformly written as follows:

$$\forall i \in [n], j \in [m], \quad b'_{ij} = \min\{d_{ij} + r_{ij}, a'_{ij}\}.$$

It then follows from (??) that

$$\sum_{j=1}^m b'_{lj} > \sum_{j=1}^m a_{lj}, \quad (3.9)$$

which means that J_l 's aggregate allocation increases from A to B' .

By definition, A 's aggregate allocation vector $\vec{A}$ is the max-min fair vector over all possible allocation matrices including those under the real demand matrix $D + R$. Based on the definition of max-min fairness and (??), there must be another job J_{i_1} whose aggregate allocation in A is no larger than J_l and whose aggregate allocation decreases from A to B' :

$$\sum_{j=1}^m a_{lj} \geq \sum_{j=1}^m a_{i_1j} > \sum_{j=1}^m b'_{i_1j}.$$

Clearly, J_{i_1} is different from J_l . Thus, by the above construction, J_{i_1} 's allocations in B' and A' are the same. Hence, the inequality above can be rewritten as:

$$\sum_{j=1}^m a_{lj} \geq \sum_{j=1}^m a_{i_1j} > \sum_{j=1}^m a'_{i_1j}. \quad (3.10)$$

Next, we construct another allocation matrix B based on A assuming that the demand matrix is $D' + R$ (which is the demand matrix used to derive A'):

- (i) For each job $J_i \neq J_l$, let $b_{ij} = a_{ij}$.
- (ii) For job J_l , if its allocation a_{ij} on site S_j does not exceed the demand $d'_{lj} + r_{lj}$, it is kept the same in B : $b_{lj} = a_{lj}$.
- (iii) For job J_l , if its allocation a_{ij} on site S_j exceeds the demand $d'_{lj} + r_{lj}$, we shrink it to be the latter demand in B : $b_{lj} = d'_{lj} + r_{lj}$.

In short, B is built by shrinking J_l 's allocation in A exceeding the demand in $D' + R$ to the latter:

$$\forall i \in [n], j \in [m], b_{ij} = \min\{d'_{ij} + r_{ij}, a_{ij}\}.$$

Since J_{i_1} is different from J_l , J_{i_1} 's allocations in B and A are the same: $\sum_{j=1}^m a_{i_1j} = \sum_{j=1}^m b_{i_1j}$. Together with (??), it can be inferred that J_{i_1} 's aggregate allocation

increases from A' to B , *i.e.*,

$$\sum_{j=1}^m b_{i_1j} > \sum_{j=1}^m a'_{i_1j}. \quad (3.11)$$

Note that both A' and B are based on the demand matrix $D' + R$ and are feasible and rational. By definition, the aggregate allocation vector of A' is the max-min fair vector over all possible allocation matrices including those under the demand matrix $D' + R$. Since J_{i_1} 's aggregate allocation increases from A' to B , based on the definition of max-min fairness, there must be another job J_{i_2} whose aggregate allocation in A' is no larger than J_{i_1} and whose aggregate allocation decreases from A' to B :

$$\sum_{j=1}^m a'_{i_1j} \geq \sum_{j=1}^m a'_{i_2j} > \sum_{j=1}^m b_{i_2j}. \quad (3.12)$$

Clearly, J_{i_2} is different from J_{i_1} . In addition, by (??), (??) and (??), we have

$$\sum_{j=1}^m a'_{l_j} > \sum_{j=1}^m a'_{i_1j} \geq \sum_{j=1}^m a'_{i_2j}. \quad (3.13)$$

Thus, J_{i_2} is also different from J_l . Hence, J_l , J_{i_1} and J_{i_2} are three distinct jobs.

By the construction of B , J_{i_2} 's allocations in B and A are the same. Moreover, by the construction of B' , J_{i_2} 's allocations in B' and A' are also the same. Therefore, (??) implies that:

$$\sum_{j=1}^m b'_{i_2j} > \sum_{j=1}^m a_{i_2j}, \quad (3.14)$$

which means that J_{i_2} 's aggregate allocation increases from A to B' .

Now we can repeat the above analysis. Similar to the argument following (??), based on the definition of max-min fairness for A and (??), there must be another job J_{i_3} whose aggregate allocation in A is no larger than J_{i_2} and whose aggregate allocation decreases from A to B' :

$$\sum_{j=1}^m a_{i_2j} \geq \sum_{j=1}^m a_{i_3j} > \sum_{j=1}^m b'_{i_3j}.$$

Similarly, it can be shown that J_{i_3} is different from any of the jobs J_l , J_{i_1} and J_{i_2} . Since J_{i_3} is different from J_l , its allocations keep the same from A to B and from A' to B' . Therefore, J_{i_3} 's aggregate allocation increases from A' to B :

$$\sum_{j=1}^m b_{i_3j} > \sum_{j=1}^m a'_{i_3j}. \quad (3.15)$$

Then, similar to the argument following (??), based on the definition of max-min fairness for A' and (??), there must be another job J_{i_4} whose aggregate allocation in A' is no larger than J_{i_3} and whose aggregate allocation decreases from A' to B :

$$\sum_{j=1}^m a'_{i_3j} \geq \sum_{j=1}^m a'_{i_4j} > \sum_{j=1}^m b_{i_4j}.$$

Similar to (??), it can be shown that J_{i_4} is different from any of the jobs J_l , J_{i_1} , J_{i_2} , J_{i_3} , and

$$\sum_{j=1}^m a'_{lj} > \sum_{j=1}^m a'_{i_1j} \geq \sum_{j=1}^m a'_{i_2j} > \sum_{j=1}^m a'_{i_3j} \geq \sum_{j=1}^m a'_{i_4j}. \quad (3.16)$$

Since J_{i_4} is different from J_l , its allocations keep the same from A to B and from A' to B' . Therefore, J_{i_4} 's aggregate allocation increases from A to B' :

$$\sum_{j=1}^m b'_{i_4j} > \sum_{j=1}^m a_{i_4j}.$$

By repeating the above process continuously, we can obtain an infinite sequence of jobs $J_{i_1}, J_{i_2}, J_{i_3}, J_{i_4}, J_{i_5}, J_{i_6}, \dots$ with non-increasing aggregate allocations in A' like (??), which contradicts that there are a finite number of jobs. Hence, the theorem is proven. $\square$

We now design an iterative algorithm (Algorithm ??) to compute the DAMF allocation, inspired by the max-min programming method from [?] which can find the max-min fair allocation vector for any max-min achievable set. The basic idea is to simultaneously increase the allocations of all jobs. Once a job's demand is completely satisfied, the algorithm fixes its allocation and continues to increase the allocations of other jobs. The algorithm terminates when all job demands are satisfied or all available resources are used up.

Algorithm 1: Computing DAMF Allocation

Input: A set of sites $\{S_1, S_2, \dots, S_m\}$ with capacities and transfer limits $\{c_1, c_2, \dots, c_j\}$; a set of jobs $\{J_1, J_2, \dots, J_n\}$ with demand matrix D ;

Output: A DAMF allocation matrix A ; a transmission matrix ΔD ;

$A \leftarrow \mathbf{0}$;

$\Delta D \leftarrow \mathbf{0}$;

$\mathcal{J} \leftarrow \{J_1, J_2, \dots, J_n\}$;

while $\mathcal{J} \neq \emptyset$ **do**

Solve the following linear program::

Maximize T , subject to::

$\forall J_i \in \mathcal{J} : \sum_{j=1}^m a_{ij} \geq T$;;

$\forall J_i \notin \mathcal{J} : \sum_{j=1}^m a_{ij} = t_i$;;

$\forall j \in [m] : \sum_{i=1}^n a_{ij} \leq u_j$;;

$\forall i \in [n], j \in [m] : 0 \leq a_{ij} \leq d_{ij} + \Delta d_{ij}$;;

$\forall j \in [m] : \sum_{i=1}^n |\Delta d_{ij}| \leq c_j$;;

$\forall i \in [n] : \sum_{j=1}^m \Delta d_{ij} = 0$;;

Let T_{max} be the outcome of the above linear program;;

for each $J_l \in \mathcal{J}$ **do**

if $\sum_{j=1}^m d_{lj} = T_{max}$ **then**

$t_l \leftarrow T_{max}$;

$\mathcal{J} \leftarrow \mathcal{J} \setminus \{J_l\}$;

else

if $\sum_{j=1}^m a_{lj} = T_{max}$ and $(\forall j \in [m], a_{lj} = d_{lj} \vee \sum_{i=1}^n a_{ij} = u_j)$ **then**

Check the feasibility of the following program::

$\sum_{j=1}^m a_{lj} > T_{max}$;;

$\forall J_i \in \mathcal{J} \wedge i \neq l : \sum_{j=1}^m a_{ij} = T_{max}$;;

$\forall J_i \notin \mathcal{J} : \sum_{j=1}^m a_{ij} = t_i$;;

$\forall j \in [m] : \sum_{i=1}^n a_{ij} \leq u_j$;;

$\forall i \in [n], j \in [m] : 0 \leq a_{ij} \leq d_{ij} + \Delta d_{ij}$;;

$\forall j \in [m] : \sum_{i=1}^n |\Delta d_{ij}| \leq c_j$;;

$\forall i \in [n] : \sum_{j=1}^m \Delta d_{ij} = 0$;;

if the above program is infeasible then

$t_l \leftarrow T_{max}$;

$\mathcal{J} \leftarrow \mathcal{J} \setminus \{J_l\}$;

return $A_{n \times m}, \Delta D_{n \times m}$;

Each iteration of our algorithm consists of two stages. In stage 1, we define a unified lower bound T and maximize it by increasing the aggregate allocations of all jobs in set $\mathcal{J}$ simultaneously, where $\mathcal{J}$ is the set of jobs whose aggregate allocations have not been fixed yet. Initially, $\mathcal{J}$ contains all jobs (line 3). The maximum T value is derived by solving a linear program which models the constraints of resource allocation introduced in Section ?? (lines 5-12), where t_i is the aggregate allocation of a job J_i that has been fixed.

Let T_{max} denote the maximum T value obtained in stage 1. In stage 2, we identify jobs whose aggregate allocations are to be fixed. If a job J_l 's total demand is equal to T_{max} amount of resources, it is fully satisfied. Thus, the algorithm fixes its aggregate allocation to be T_{max} and removes it from $\mathcal{J}$ (lines 15-17). Otherwise, the algorithm checks whether it is possible to increase J_l 's aggregate allocation beyond T_{max} without violating other jobs' allocations. If J_l 's aggregate allocation from stage 1 is already larger than T_{max} , or there exists a site on which J_l 's demand is not fully satisfied and the site has spare capacity (i.e., the condition in line 18 is not satisfied), clearly J_l 's aggregate allocation can be further increased. Otherwise, we change the aggregate allocation constraints for J_l and other jobs by replacing " $\geq$ " with " $>$ " (line 20) and " $=$ " (line 21) respectively, and then check whether the new linear program has any feasible solution (lines 19-26). If so, it implies that J_l 's allocation can be increased and J_l will be kept in set $\mathcal{J}$. Otherwise, J_l 's aggregate allocation is fixed at T_{max} and J_l is removed from set $\mathcal{J}$ (lines 27-29). Note that this T_{max} value is the aggregate resource amount, which means that J_l 's allocations on individual sites could vary in subsequent iterations as long as the aggregate amount is kept. The algorithm terminates when $\mathcal{J}$ becomes empty (line 4).

The linear program in Algorithm ?? has $2mn$ variables and $2m + 2n + mn$ constraints, where m and n are the numbers of sites and jobs respectively. Since there can be up to n iterations and each iteration needs to solve up to n linear programs in stage 2, the worst-case time complexity of Algorithm ?? is $O(n^2 * LP(2mn, 2m + 2n + mn))$, where $LP(2mn, 2m + 2n + mn)$ is the complexity for solving a linear program with $2mn$ variables and $2m + 2n + mn$ constraints.

3.4.4 A Fast Approximation

Algorithm 1 computes an exact DAMF allocation. However, its complexity could raise concern since solving large-scale linear programs can be computationally expensive [?]. In this section, we propose a heuristic algorithm to derive a near-DAMF allocation with significantly reduced time complexity to enhance the practicality.

The heuristic algorithm (Algorithm ??) proceeds in two steps. In step 1 (lines 4-9), we find a fast approximation to AMF (i.e., without transferring job demands between sites) in an iterative manner. Each iteration calculates the compute resource allocation for one site S_j , given the job demands and compute capacity on this site as well as the aggregate allocation vector $\vec{p}$ over the sites that have been processed in previous iterations (line 5). Suppose there are n jobs running in the system. Let $\vec{p} = \langle p_1, p_2, \dots, p_n \rangle$ be the current aggregate allocation vector and $\langle d_{1j}, d_{2j}, \dots, d_{nj} \rangle$ be the job demands on site S_j . We would like to compute an allocation $\langle a_{1j}, a_{2j}, \dots, a_{nj} \rangle$ on S_j so that $\langle p_1 + a_{1j}, p_2 + a_{2j}, \dots, p_n + a_{nj} \rangle$ is max-min fair among all possible allocations for the given $\vec{p}$. If the total demand of all jobs on site S_j is no larger than its capacity, i.e., $\sum_{i=1}^n d_{ij} \leq u_j$, we can simply satisfy the demands of all jobs by setting $a_{ij} = d_{ij}$ for all i 's. Otherwise, we sort the jobs in increasing order of p_i , i.e., $p_1 \leq p_2 \leq \dots \leq p_n$, and the resources on site S_j will be allocated to a prefix of this job sequence by the definition of max-min fairness. If resources are allocated to the first k jobs, the total resources allocated must be no more than

$$\sum_{i=1}^k (\min\{p_{k+1}, p_i + d_{ij}\} - p_i), \quad (3.17)$$

because the aggregate resource allocation of each job cannot exceed the given allocation p_{k+1} of the next job ($(k+1)$ -th job). Since (??) is non-decreasing with k , we can use a binary search to find out which jobs will receive resource allocations on S_j to make full use of its capacity u_j . After deciding k , we solve the equation $\sum_{i=1}^k (\min\{C, p_i + d_{ij}\} - p_i) = u_j$ to find out the maximum aggregate resource allocation C among these jobs to achieve max-min fairness. Then, the i -th job will be allocated $a_{ij} = \min\{C, p_i + d_{ij}\} - p_i$ amount of resources. Apparently, the iterating order of sites has an impact on the final allocation result. Our idea is to sort all sites in increasing order of the number of jobs with demands on the site (line 3).

Algorithm 2: Heuristic to Approximate DAMF Allocation

Input: A set of sites $\{S_1, S_2, \dots, S_m\}$ with capacities and transfer limits; a set of jobs $\{J_1, J_2, \dots, J_n\}$ with demand matrix D ;

Output: An allocation matrix A ; a transmission matrix ΔD ;

$A = \mathbf{0}$;

$\Delta D = \mathbf{0}$;

$\vec{p} = \mathbf{0}$;

Sort all sites in increasing order of the number of jobs with demands on each site (i.e., the cardinality of $\{J_i : a_{ij} > 0\}$);

for each site S_j in the sorted order **do**

 Compute max-min fair allocation $\langle a_{1j}, a_{2j}, \dots, a_{nj} \rangle$ on S_j based on job demands $\{d_{1j}, d_{2j}, \dots, d_{nj}\}$, capacity u_j of S_j , and current aggregate allocation vector $\vec{p} = \langle p_1, p_2, \dots, p_n \rangle$ as described in the main text;

for $i \in [n]$ **do**

$p_i = p_i + a_{ij}$;

$\mathcal{S}_{\text{spare}} = \{S_j \mid \sum_{i=1}^n a_{ij} < u_j\}$;

$\mathcal{S}_{\text{full}} = \{S_j \mid \sum_{i=1}^n d_{ij} > u_j\}$;

$\mathcal{J} = \{J_i \mid \sum_{j=1}^m d_{ij} > \sum_{j=1}^m a_{ij}\}$;

while $\mathcal{S}_{\text{spare}} \neq \emptyset \wedge \mathcal{S}_{\text{full}} \neq \emptyset \wedge \mathcal{J} \neq \emptyset$ **do**

 Remove all S_l 's from $\mathcal{S}_{\text{spare}}$ and $\mathcal{S}_{\text{full}}$ that $c_l = 0$ or $\sum_{i=1}^n (d_{il} + \Delta d_{il}) = u_l$;

while $\mathcal{J} \neq \emptyset$ **do**

 Select job $J_x \in \mathcal{J}$ with the least aggregate allocation;

if $\forall S_j \in \mathcal{S}_{\text{full}}, d_{xj} + \Delta d_{xj} \leq a_{xj}$ **then**

 Remove J_x from $\mathcal{J}$;

else

 Select a site $S_y \in \mathcal{S}_{\text{full}}$ where $d_{xy} + \Delta d_{xy} > a_{xy}$;

 Select a site $S_z \in \mathcal{S}_{\text{spare}}$;

$\Delta d_{xy} = \Delta d_{xy} - 1$;

$\Delta d_{xz} = \Delta d_{xz} + 1$;

$a_{xz} = a_{xz} + 1$;

$c_y = c_y - 1$;

$c_z = c_z - 1$;

return $A_{n \times m}, \Delta D_{n \times m}$;

A site demanded by more jobs is considered to have higher potential in balancing the final aggregate allocation, so we process it in a later iteration.

In step 2 (lines 10-30), we utilize the network resources to improve the allocation derived in step 1. It is not difficult to see that there exists a DAMF allocation (represented by a pair of transmission and allocation matrices) such that for every demand unit (one task) transferred, (i) the task involved is allocated compute resources at the destination site; and (ii) the compute resources at the source site are fully allocated. In fact, if a demand unit transferred does not satisfy (i), the transfer can be canceled without affecting the allocation matrix. Similarly, the transfer of a demand unit that violates (ii) can also be canceled and the task can be satisfied by compute resources at the source site. This would increase the compute resource allocation of the job involved without affecting the compute resource allocation of any other job, thereby contradicting that the original allocation is max-min fair.

Based on the observation above, we define two sets $\mathcal{S}_{spare}$ and $\mathcal{S}_{full}$ to record sites with spare capacities and with more job demands than capacities (lines 10-11), and a set $\mathcal{J}$ to record jobs whose demands are not fully satisfied (line 12). Then, we start a loop to transfer job demands from $\mathcal{S}_{full}$ sites to $\mathcal{S}_{spare}$ sites until either $\mathcal{S}_{spare}$, $\mathcal{S}_{full}$ or $\mathcal{J}$ becomes empty (which implies that no further improvement to the allocation can be made). To make effective use of network resources, we would like all job demands transferred to get compute resource allocations at destination sites. Intuitively, a site will not need to participate in transmissions if its bandwidth is used up or its updated local demand equals its capacity. Thus, within the loop, we first remove all such sites from $\mathcal{S}_{spare}$ and $\mathcal{S}_{full}$ (line 14). As a heuristic to approximate max-min fairness, among all jobs whose demands are not completely satisfied, we pick the job J_x with the least aggregate allocation currently (line 16). If J_x 's demands are satisfied on all sites in $\mathcal{S}_{full}$ (implying that J_x 's unsatisfied demands locates at the sites that have been removed because no more bandwidth is available), we discard J_x and examine the next job with the least aggregate allocation for demand transmission (lines 17-18). Otherwise, we select the source site and destination site for a transfer of J_x : the source site can be any site in $\mathcal{S}_{full}$ where J_x 's demand is not fully satisfied (line 20), and the destination site can be any site in $\mathcal{S}_{spare}$ (line 21). After deciding a transmission, we update the related matrices and remaining transfer limits (lines 22-26).

The heuristic algorithm does not require solving linear programs and has much lower time complexity. In step 1, it solves m allocation problems separately (where m is the number of sites). In the water-filling-like strategy, sorting takes $O(n \log n)$ time (where n is the number of jobs), and the binary search takes $O(n \log n)$ time as computing (??) takes $O(n)$ time. So, the time complexity of each iteration is $O(n \log n)$, and the total time complexity of step 1 is $O(mn \log n)$. In step 2, the algorithm transfers at most $cm/2 = O(m)$ units of job demand (assuming the transfer limits of all sites are a constant c). Updating $\mathcal{S}_{\text{spare}}$ and $\mathcal{S}_{\text{full}}$ takes $O(m)$ time. If we arrange all unsatisfied jobs in $\mathcal{J}$ as a min-heap based on their aggregate allocations, selecting the unsatisfied job with the least aggregate allocation takes $O(1)$ time. Selecting the source and destination sites takes $O(m)$ time. After the demand transfer, updating the min-heap of $\mathcal{J}$ takes $O(\log n)$ time. Thus, each unit of demand transfer takes $O(m + \log n)$ time. If a source site for the selected job can always be found, the time complexity of step 2 is $O(m(m + \log n))$. If the selected job fails to find a source site for transmission, the job is discarded and updating the min-heap of $\mathcal{J}$ takes $O(\log n)$ time. In the worst case, discarding all such jobs incur a time complexity of $O(n(m + \log n))$. Therefore, the worst-case time complexity of step 2 is $O((m + n)(m + \log n))$. Hence, the overall complexity of the algorithm is $O(mn \log n + m^2)$. In the experiments, we shall empirically evaluate and compare the computation times of Algorithms ?? and ??.

3.4.5 Optimizing Job Completion Times

In addition to providing fair resource allocation from the global perspective, we can further optimize the job completion times. The key idea is that DAMF fixes only the aggregate resource allocation for each job, while the particular allocations at individual sites can still be adjusted. In this case, the completion time of a job can be optimized by assigning the allocation proportionally to its demand at each site. For example, if a job has 2 and 4 tasks at two sites respectively and its aggregate allocation by DAMF is 3 computing slots, then its allocations at the two sites can be set to 1 and 2 computing slots, so that the two sites can finish processing their tasks at the same time (to minimize job completion time) if all tasks have the same processing time. Meanwhile, we need to ensure that other jobs can also achieve their DAMF aggregate allocations. The details of our algorithm are shown in Algorithm ??.

Algorithm 3: Optimizing Job Completion Times

Input: A set of sites $\{S_1, S_2, \dots, S_m\}$ with capacities; a set of jobs $\{J_1, J_2, \dots, J_n\}$ with demand matrix D ; the aggregate allocation e_i of each job J_i in a DAMF allocation with corresponding transmission matrix ΔD ;

Output: An optimized allocation matrix A' ;

```

for  $k = 1, 2, \dots, n$  do
    Solve the following linear program;;
    Maximize  $y_k$ , subject to;;
     $\forall j \in [m], \sum_{i=1}^n a_{ij} \leq u_j$ ;;
     $\forall i < k, j \in [m] : a_{ij} = a'_{ij}$ ;;
     $\forall i \geq k, j \in [m] : 0 \leq a_{ij} \leq d_{ij} + \Delta d_{ij}$ ;;
     $\forall i \geq k : \sum_{j=1}^m a_{ij} = e_i$ ;;
     $\forall j \in [m] : a_{kj} = d_{kj} * y_k + z_{kj}, y_k \geq 0, z_{kj} \geq 0$ ;;
    for each  $j \in [m]$  do
         $a'_{kj} \leftarrow a_{kj}$ ;
return  $A'_{n \times m}$ ;

```

Algorithm ?? optimizes the completion times of jobs iteratively from J_1 to J_n . In each iteration, the algorithm solves a linear program (lines 4-10) to optimize the completion time of a job J_k . The first constraint of the linear program (line 6) ensures that the allocations are feasible. The second constraint (line 7) retains the allocations of jobs already optimized. The third constraint (line 8) ensures that the allocations of remaining jobs are rational given the input transmission matrix. The fourth constraint (line 9) guarantees that the aggregate allocations of these jobs comply with DAMF. The last constraint (line 10) expresses J_k 's allocation a_{kj} at each site S_j as a linear combination $d_{kj} * y_k + z_{kj}$, where y_k is a shared proportional coefficient for all sites and z_{kj} is an auxiliary variable. Thus, by maximizing y_k , the resource allocation a_{kj} across sites can be made as close to proportional as possible to the resource demand $d_{kj} + \Delta d_{kj}$ at each site, thereby minimizing job J_k 's completion time. After solving the linear program, J_k 's allocation at each site is decided and recorded in A' (lines 11-13). Finally, the algorithm returns the optimized allocation A' after all iterations. The set of jobs $\{J_1, J_2, \dots, J_n\}$ input to the algorithm can be arranged in different orders based on job arrival times, job priorities, job demands, etc. In this paper, we sort the jobs based on their total demands from low to high so that jobs with smaller numbers of tasks are optimized earlier.

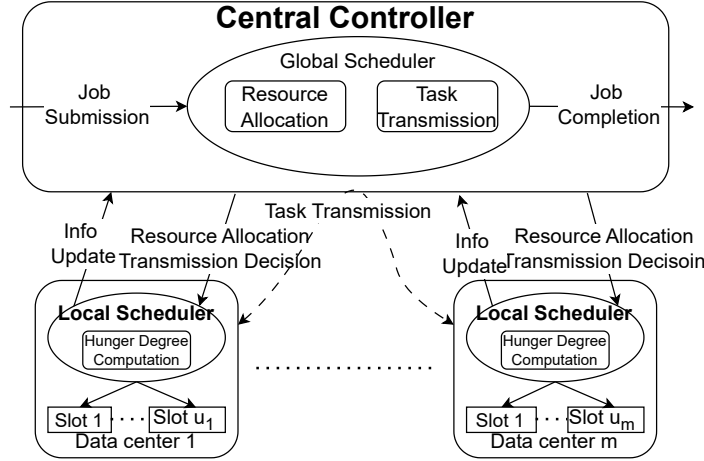


FIGURE 3.2: System Architecture

3.4.6 Deployment

Algorithms ?? and ?? in the above sections derive a DAMF or near-DAMF resource allocation among active jobs at a snapshot of the system. In this section, we discuss how to apply them to online scenarios where jobs arrive dynamically for execution. As shown in Figure ??, the algorithm for computing the DAMF allocation can be deployed at a central scheduler of the distributed job execution system. The scheduler tracks the real-time resource capacities of the sites and the distributed demands of each job on all sites. Once an allocation is derived, the scheduler propagates the transmission matrix ΔD and the allocation matrix A to relevant sites, where job demands (task input data) would be transferred based on ΔD . The compute resource allocation at each site would follow the allocation matrix A . If the demand of a job on a site is larger than its allocation on the site, some of its tasks will have to wait. When a computing slot becomes available on a site, the site computes the "Hunger Degree" for jobs with pending tasks locally and selects the job with the lowest Hunger Degree to launch its task in the available slot. More specifically, the Hunger Degree of a job on a site is the ratio between the number of its tasks that are currently running (the number of computing slots occupied by this job now) and its DAMF allocation on this site (the number of computing slots that this job is supposed to occupy). A lower Hunger Degree indicates that the job is taking up less slots than expected and deserves more slot allocation. In particular, if a job's DAMF allocation at a site is zero, its Hunger Degree is set to infinity.

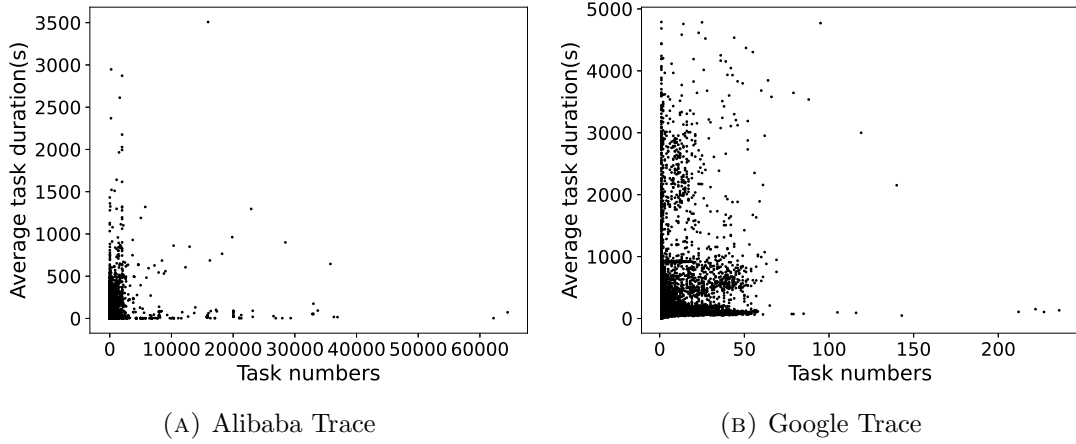


FIGURE 3.3: Task characteristics of two datasets

TABLE 3.2: Dataset characteristics

Dataset	Average task duration	Average task count per job
Alibaba	155.68 seconds	273.5 tasks
Google	512.25 seconds	4.5 tasks

In distributed job execution, the demands of jobs change continuously because of job arrivals and task completions. In general, the completion of a single task would not significantly change the relative distribution of job demands and hence the resultant resource allocation [?]. Therefore, to reduce the computational overhead, allocations can be recomputed only when a new job arrives or all tasks of an existing job are completed on some site. Recall that in our model, there is a time threshold T which determines the limit c_j on the total demand that can be transferred into and out of each site S_j . This parameter T can be dynamically adjusted according to the job arrival rate. When new jobs arrive frequently, T can be shortened to align with the reduced time between recomputations; conversely, T can be lengthened if the job arrival rate decreases.

3.5 Experiments

Simulation experiments are conducted to compare the fairness and efficiency between various allocation policies.

3.5.1 Datasets

We use job traces from Alibaba [?] and Google [?]. The Alibaba trace contains information on approximately 16 million tasks collected from 1300 machines over a period of 12 hours in August 2017. The Google trace records events (e.g., a task is scheduled, completed or aborted) from Google’s eight Borg cells over the month of May 2019. We pre-process the raw traces to extract key information, including the arrival time, task count and task durations of each job. From each trace, a continuous sequence of 15,000 jobs is randomly selected to serve as the dataset driving our simulations.

Figure ?? and Table ?? illustrate the task characteristics of both datasets, including the task number and mean task duration of each job, and the average task duration and task count of all jobs. It can be seen that the jobs in the Alibaba dataset generally have larger numbers of tasks and shorter average task durations, whereas the jobs in the Google dataset tend to have longer average task durations and significantly fewer tasks.

3.5.2 System Configuration

We simulate a distributed system of m sites, each with 32 computing slots. Each task is assumed to take up one computing slot to run and release it after completion. Due to the switching overhead consideration, we assume that there is no preemption during the execution of a task. For each job, the distribution of its tasks among the sites is generated by a Zipf distribution: the numbers of tasks placed on the sites are proportional to $1, \frac{1}{2^\alpha}, \frac{1}{3^\alpha}, \dots, \frac{1}{m^\alpha}$, where α is a Zipf parameter. A higher α implies a more skewed distribution (a higher proportion of tasks are concentrated at one site), whereas $\alpha = 0.0$ indicates a uniform distribution. In the experiments, we test four Zipf distributions with α set to 0.0, 0.5, 1.0 and 2.0 to simulate different workload situations. We shuffle the order of the sites when deploying each job’s task distribution, so that the overall workload distribution of all jobs is roughly balanced among sites. We scale the job arrival times to simulate different levels of overall system workloads. We experiment with different numbers of sites and different workload levels, and observe similar performance trends. We implement the algorithms and run the experiments on a server with AMD EPYC 7H12 2.6GHz processor and 1024GB memory.

3.5.3 Resource Allocation Policies

Besides Algorithms ?? and ?? (called DAMF and DAMF_a for short), we also implement three baseline policies: IMF, AMF (both introduced in Section ??) and Dynamic Independent Max-min Fairness (DIMF). DIMF utilizes network resources on top of IMF: it first applies the IMF policy for resource allocation on each site, then transfers unsatisfied job demands to the sites with spare compute capacities to maximize the system utility (subject to the transfer limits), where jobs with lower aggregate allocations in the first step are assigned higher transmission priority in order to promote fairness. All the policies recompute the allocation matrix only when a new job arrives or an existing job completes all of its tasks on some site. We use CPLEX [?] as the program solver in the experiments. Since a task cannot be transferred fractionally, the transmission matrix ΔD derived to achieve the DAMF allocation may not be workable. To make DAMF applicable in practice, we enforce the elements of the transmission matrix ΔD in Algorithm ?? to be integers, i.e., replacing the linear program with a mixed-integer program. This would incur extra but acceptable computation time. We shall demonstrate the computation times of DAMF and DAMF_a later. Note that DAMF_a does not need to apply a similar modification because the transmission matrix ΔD derived from its heuristic is always integral.

As introduced in Section ??, transmission count limit c is computed based on the data partition size S , available bandwidth B and time threshold T : $c = BT/S$. In the experiments, we set $T = 50$ seconds (which is small compared to the average task duration shown in Table ??), $S = 1\text{GB}$ and $B = 160\text{Mbps}$ or 480Mbps for all sites, thus $c = (160 \text{ or } 480) * 50 / (1 * 1024 * 8) \approx 1 \text{ or } 3$. Under these settings, a task transmission takes $50 / (1 \text{ or } 3) = 50 \text{ or } 16.67$ seconds to arrive at its destination. To emulate the bandwidth bottleneck being the connection between each site and the backbone network, task transmissions (potentially originating from different source sites) are received sequentially at each destination site. In recomputing the allocation and transmission matrices, the scheduler discards all pending transmissions (but retains the ongoing one) to release more bandwidth for the reallocation. For example, suppose $c = 3$ for all sites and a recomputation is triggered 30 seconds after the previous one. Suppose in the previous computation, it was decided that a site S_j would receive 3 tasks. In this case, the first task transmission would have arrived at site S_j 16.67 seconds after the previous computation, while the

second task transmission is ongoing and the third one is pending. Then, in the recomputation, the third task transmission will be rescinded and the transmission count limit c_j available for S_j will be set to 2.

3.5.4 Performance Metrics

In Sections ?? and ??, we have proved that DAMF achieves several important fairness properties. Our experiments aim to further evaluate DAMF and DAMF_a's empirical performance from different perspectives. We study the following performance metrics: the deviation of the instantaneous resource allocation from ideal DAMF, the standard deviation of the instantaneous resource allocation among jobs, the average Jain's Index of instantaneous resource allocation, and the average job response time.

The deviation of the instantaneous resource allocation measures the instantaneous difference between the ideal DAMF allocation and the actual allocation of a policy. We continuously trace the number of computing slots actually occupied by each job throughout the simulation of different policies and compare it with the ideal DAMF allocation. More specifically, we first conduct the simulation using DAMF, and at each recomputation within simulation, we record the output transmission and allocation matrices as well as the resultant aggregate allocation vector with timestamp, which is taken as the ideal DAMF allocation. We also track the actual aggregate allocation of DAMF by updating the actual aggregate allocation vector with timestamp when a computing slot is allocated to a job (to run a task) or released by a job (when a task is completed). Note that available computing slots are assigned by the Hunger Degree mechanism and there is no preemption. Thus, the actual instantaneous allocations (resp. aggregate allocations) may differ from the allocation matrices (resp. aggregate allocation vectors) output by recomputations. Next, we perform simulations of other policies and record their actual allocations as well. Finally, we compute and trace the allocation deviation between an actual allocation and the ideal DAMF allocation over the simulated time span. Given two aggregate allocation vectors $\vec{A} = \langle a_1, a_2, \dots, a_n \rangle$. and $\vec{B} = \langle b_1, b_2, \dots, b_n \rangle$, the allocation deviation is computed as $\sum_{i=1}^n |a_i - b_i|$. The allocations of completed jobs are treated as 0. A lower deviation indicates a closer instantaneous allocation to the ideal DAMF allocation. We plot the cumulative distribution of the deviation over

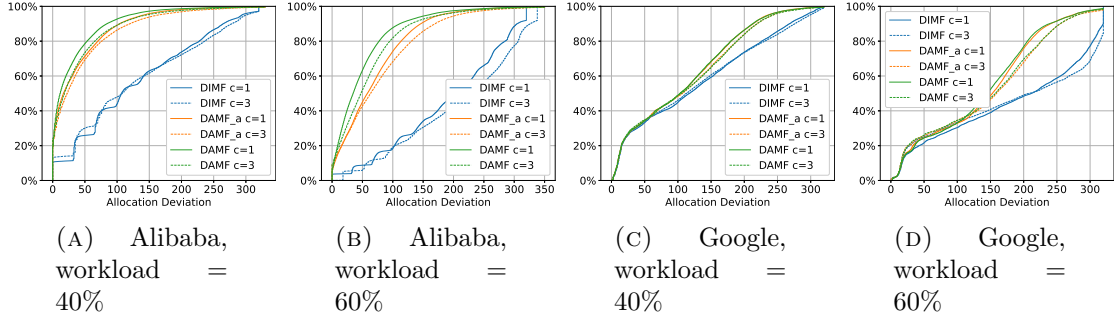


FIGURE 3.4: Cumulative distribution of allocation deviation, Zipf parameter $\alpha = 2$

the simulated time span. If the distribution curve is closer to the upper left corner, the allocation policy is considered to have better instantaneous fairness, because it has a lower deviation for a higher percentage of time.

The standard deviation of the instantaneous resource allocation among jobs is computed from the records of the actual aggregate allocation vectors of jobs. We exclude jobs that have completed or not yet arrived from the computation of standard deviation. A lower standard deviation naturally indicates a more balanced resource allocation among jobs. We plot the cumulative distribution of the standard deviation over the simulated time span for comparison. Again, allocation policies whose distribution curves are closer to the upper left corner are considered superior in terms of instantaneous fairness.

The Jain's Index is a quantitative measure used to assess the fairness or equity of resource allocation among n competing users or entities. The range of Jain's Index is $[\frac{1}{n}, 1]$, and a higher index indicates a more balanced allocation. Suppose each user i gets x_i resources, then Jain's Index can be computed as follows:

$$f(x_1, x_2, \dots, x_n) = \frac{(\sum_{i=1}^n x_i)^2}{n * \sum_{i=1}^n x_i^2}.$$

We compute the Jain's Index from the records of the actual aggregate allocation vectors of jobs, and calculate the average Jain's Index over the simulated time span for comparison.

The response time of a job is the time duration from its arrival to its completion. We use the average job response time to evaluate the long-term impact of allocation policies. A lower average job response time suggests a higher system efficiency.

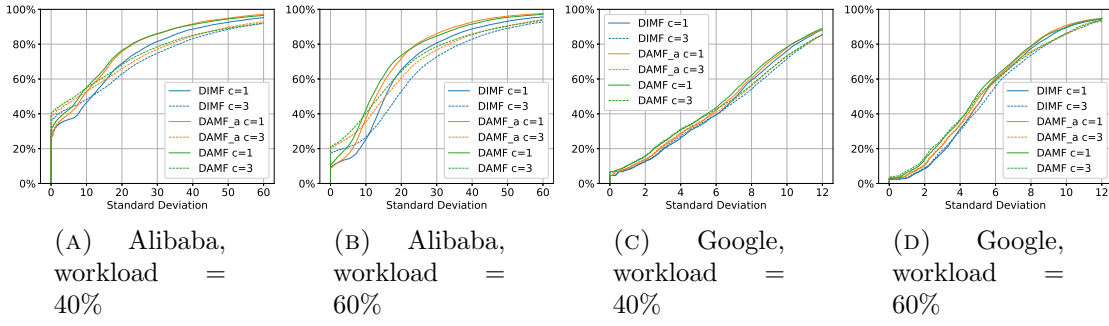


FIGURE 3.5: Cumulative distribution of standard deviation, Zipf parameter $\alpha = 2$

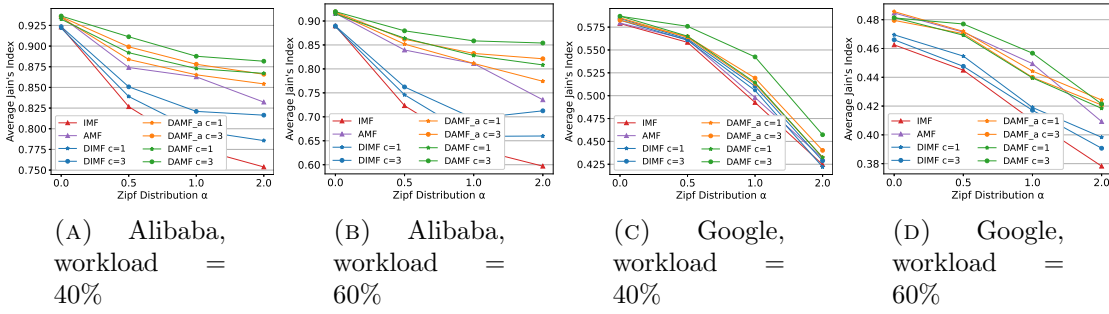


FIGURE 3.6: Average Jain's Index

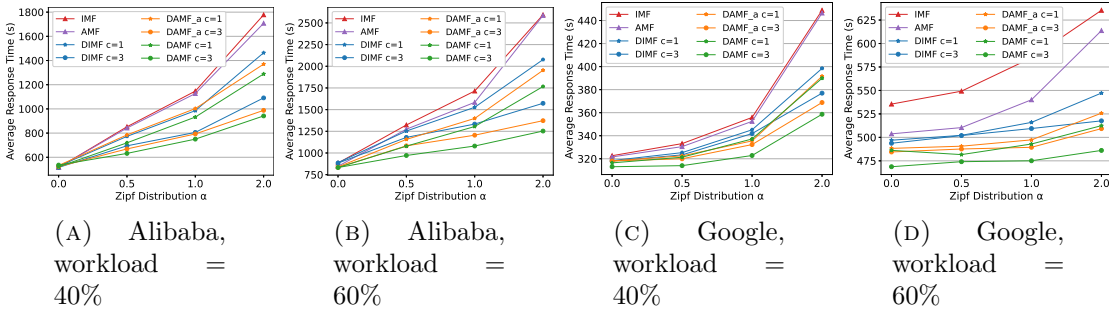


FIGURE 3.7: Average job response time

3.5.5 Experimental Results

Figure ?? shows the cumulative distribution of the allocation deviation from the ideal DAMF. Due to space limitations, we only present the results for the Zipf distribution of $\alpha = 2$. The results for other Zipf distributions show similar trends. Comparing DAMF and the baseline DIMF under the same transfer limit, it can be seen that DAMF's distribution curves are closer to the upper left corner, which indicates that DAMF achieves better instantaneous fairness than DIMF. In addition, the distribution curves of DAMF and DAMF_a are rather close to each other under the same transfer limit, demonstrating that DAMF_a is a close approximation of

TABLE 3.3: Average computation time (s), Alibaba

		workload = 40%		workload = 60%	
c	α	DAMF	DAMF_a	DAMF	DAMF_a
1	0.0	0.0747	$1.005 * 10^{-5}$	0.1287	$1.402 * 10^{-5}$
	0.5	0.0792	$9.223 * 10^{-6}$	0.1661	$1.440 * 10^{-5}$
	1.0	0.0793	$9.409 * 10^{-6}$	0.1764	$1.453 * 10^{-5}$
	2.0	0.0841	$8.205 * 10^{-6}$	0.1709	$1.401 * 10^{-5}$
3	0.0	0.0709	$8.749 * 10^{-6}$	0.1497	$1.690 * 10^{-5}$
	0.5	0.0765	$8.395 * 10^{-6}$	0.1571	$1.351 * 10^{-5}$
	1.0	0.0761	$8.853 * 10^{-6}$	0.1551	$1.424 * 10^{-5}$
	2.0	0.0837	$1.051 * 10^{-5}$	0.1733	$1.316 * 10^{-5}$

TABLE 3.4: Average computation time (s), Google

		workload = 40%		workload = 60%	
c	α	DAMF	DAMF_a	DAMF	DAMF_a
1	0.0	0.1029	$1.710 * 10^{-5}$	0.3783	$6.346 * 10^{-5}$
	0.5	0.1247	$1.836 * 10^{-5}$	0.3771	$6.084 * 10^{-5}$
	1.0	0.1298	$1.610 * 10^{-5}$	0.3952	$5.071 * 10^{-5}$
	2.0	0.1164	$1.686 * 10^{-5}$	0.3708	$3.817 * 10^{-5}$
3	0.0	0.1180	$1.508 * 10^{-5}$	0.4670	$7.205 * 10^{-5}$
	0.5	0.1400	$1.583 * 10^{-5}$	0.5175	$5.482 * 10^{-5}$
	1.0	0.1489	$1.496 * 10^{-5}$	0.5289	$5.178 * 10^{-5}$
	2.0	0.1796	$1.798 * 10^{-5}$	0.5465	$3.670 * 10^{-5}$

DAMF. We also record and compare the average computation time for recomputing allocation matrices during simulations. As shown in Table ?? and ??, DAMF_a reduces the average computation time by four orders of magnitude compared to DAMF (from the order of 10^{-1} seconds down to 10^{-5} seconds).

Comparing the results for the Alibaba and Google datasets under the same transfer limit, it can be seen that the Google dataset has much higher allocation deviations than the Alibaba dataset. This is mainly due to task durations: the Google dataset has much longer task durations than the Alibaba dataset as shown in Table ?. Since there is no preemption, an updated allocation matrix after recomputation starts to take effect only after the computing slots are released (when the currently running tasks are completed) and reallocated to the intended jobs. Longer task durations make it slower for the actual allocation to converge to the allocation matrix computed.

Figure ?? shows the cumulative distribution of the standard deviation of the instantaneous resource allocation among jobs. Due to space limitations, we again only present the results for the Zipf distribution of $\alpha = 2$. Similar to the results in Figure ??, it can be seen that DAMF's distribution curves are closer to the upper left corner, which indicates that DAMF achieves more balanced instantaneous resource allocations than DIMF. Comparing DAMF and DAMF_a under the same transfer limit, we can see that their distribution curves are quite close to each other, which again shows that DAMF_a is a close approximation of DAMF. It can also be observed from Figure ?? that the standard deviation tends to increase with the transfer limit. This is because in DAMF, small-demand jobs are prioritized to have their demands fully satisfied at their original sites, while large-demand jobs are more likely to have unsatisfied demands and tend to exploit idle compute resources at other sites by leveraging network resources. Therefore, transmissions often help large-demand jobs to obtain more resources while keeping the resources allocated to small-demand jobs unchanged, which leads to the rise of the standard deviation of the instantaneous resource allocation with increasing transfer limit.

Figure ?? shows the average Jain's Index. It can be seen that allocation policies that allow task transmissions (DAMF, DAMF_a, DIMF) have higher Jain's Indexes compared with their counterparts which do not (AMF, IMF). More importantly, DAMF achieves the highest index while DAMF_a is close to it, demonstrating that they effectively reduce the imbalance of resource allocation among jobs.

Figure ?? shows the average job response time for different datasets, system workloads and Zipf distributions. It can be seen that policies utilizing network resources achieve lower average job response time, compared with IMF and AMF. The improvement typically increases with the transfer limit. Furthermore, since DAMF views the whole distributed system as a united resource pool and requires the aggregate allocation vector to be max-min fair among all possible transmissions and allocations, it outperforms DIMF and other allocation policies in most cases. The improvement is substantial even with only one bandwidth unit available, and it becomes more significant when the system workload is higher or the task distribution of jobs is more skewed among sites. In addition, DAMF_a has similar performance of average job response time to DAMF, demonstrating that it closely approximates the original DAMF.

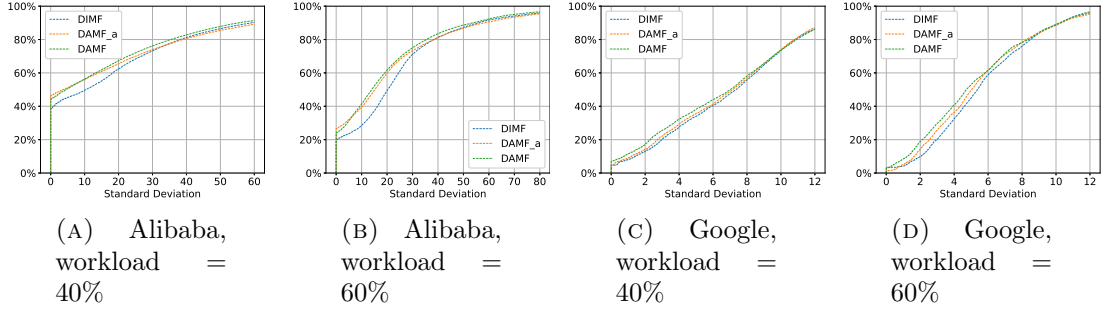


FIGURE 3.8: Cumulative distribution of standard deviation under fluctuating bandwidth, Zipf parameter $\alpha = 2$

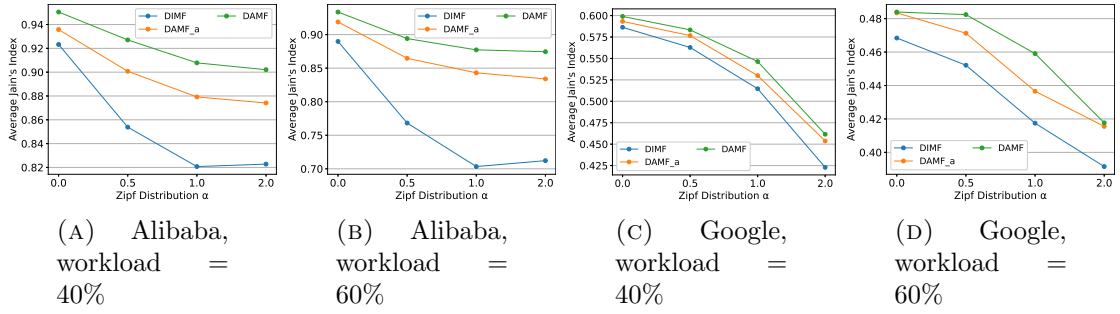


FIGURE 3.9: Average Jain's Index under fluctuating bandwidth

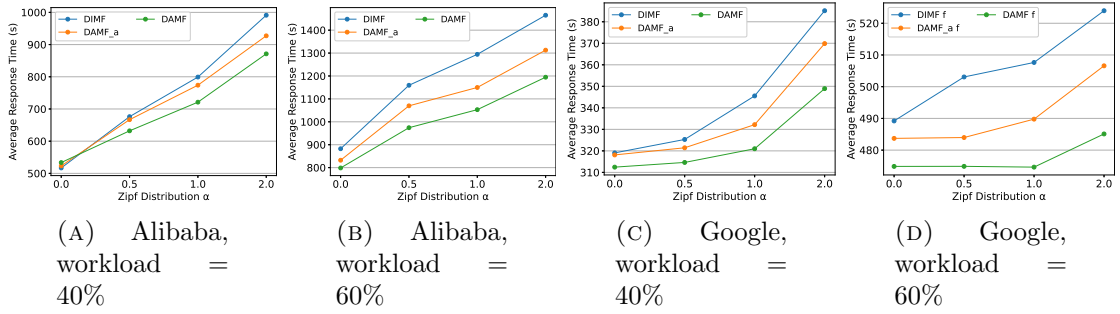


FIGURE 3.10: Average job response time under fluctuating bandwidth

3.5.6 Evaluation on Robustness and Scalability

We also conduct experiments to evaluate the allocation policies under fluctuating bandwidth conditions. Specifically, we initialize an integer sequence randomly generated from a uniform distribution between 1 and 5 to simulate the changes of the transfer count limit c (which reflects the available bandwidth) over time. In the experiments, once the i -th job arrives, the value of c for all sites is changed to the i -th element in the sequence for recomputation. For fair comparison, the same sequence is used for evaluating all allocation policies. As shown in Figures ??, ?? and ??, the performance trends of different policies (that utilize network resources)

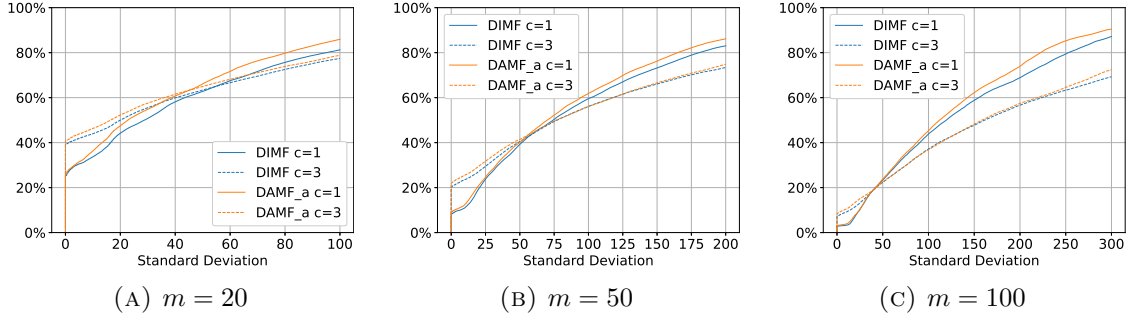


FIGURE 3.11: Cumulative distribution of standard deviation, Alibaba, workload = 40%, Zipf parameter $\alpha = 1$

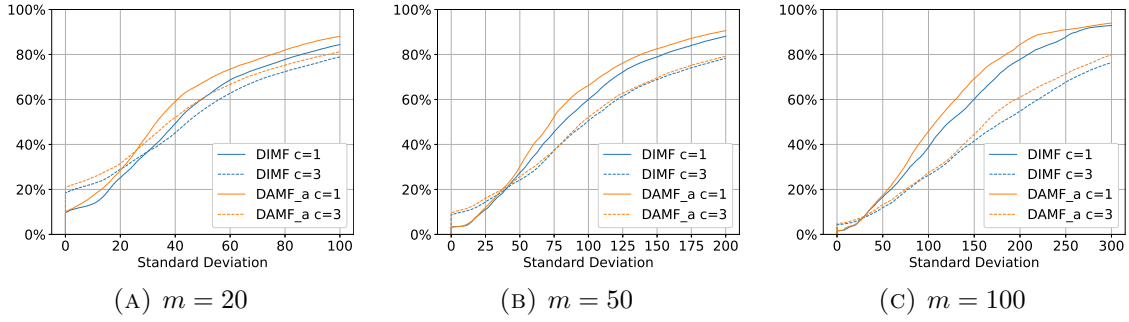


FIGURE 3.12: Cumulative distribution of standard deviation, Alibaba, workload = 60%, Zipf parameter $\alpha = 1$

remain similar to previous experiments. DAMF and DAMF_a again significantly outperform the baseline policy DIMF.

In addition, we evaluate the scalability of DAMF_a under larger numbers of sites m . We set $m = 20, 50$ and 100 and conduct experiments for DAMF_a and DIMF. Since the average task count per job of the Google dataset is low (4.5 tasks per job), it is not suitable for evaluation with a large number of sites. Thus, we test with the Alibaba dataset only. Due to space limitations, for performance metrics other than the average computation time, we present only the results for the Zipf distribution of $\alpha = 1$ in Figures ??, ??, ?? and ?? (those for other α values show similar trends). These results again demonstrate that DAMF_a outperforms the baseline DIMF in terms of both resource allocation fairness and system utility. Table ?? shows the average computation times for different site numbers and Zipf distributions. These results demonstrate that DAMF_a maintains low computational overhead, thereby validating its potential for large-scale geo-distributed systems.

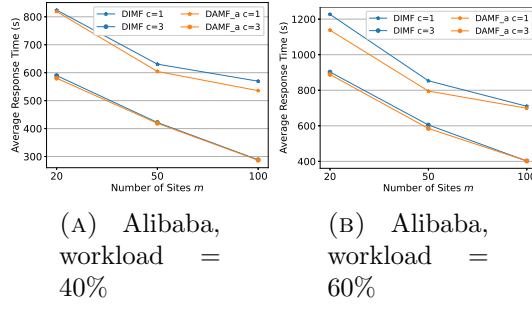
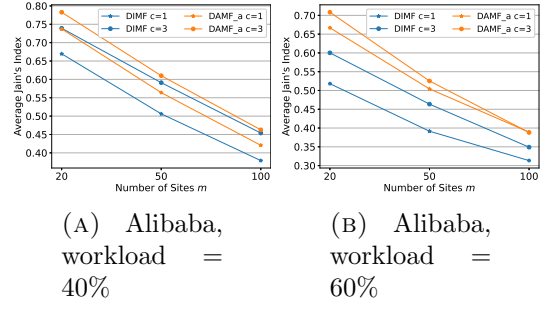

 FIGURE 3.13: Average job response time, Zipf parameter $\alpha = 1$

 FIGURE 3.14: Average Jain's Index, Zipf parameter $\alpha = 1$

 TABLE 3.5: Average computation time (s) of DAMF_a

		Alibaba, workload = 40%			Alibaba, workload = 60%		
c	α	$m = 20$	$m = 50$	$m = 100$	$m = 20$	$m = 50$	$m = 100$
1	0.0	2.035×10^{-5}	4.908×10^{-5}	8.455×10^{-5}	3.732×10^{-5}	7.321×10^{-5}	1.594×10^{-4}
	0.5	2.231×10^{-5}	4.491×10^{-5}	7.954×10^{-5}	3.339×10^{-5}	6.835×10^{-5}	1.592×10^{-4}
	1.0	1.645×10^{-5}	4.084×10^{-5}	7.899×10^{-5}	2.517×10^{-5}	6.675×10^{-5}	1.439×10^{-4}
	2.0	1.586×10^{-5}	3.561×10^{-5}	8.723×10^{-5}	2.243×10^{-5}	5.335×10^{-5}	1.219×10^{-4}
3	0.0	2.169×10^{-5}	4.678×10^{-5}	9.114×10^{-5}	3.644×10^{-5}	7.452×10^{-5}	1.639×10^{-4}
	0.5	2.195×10^{-5}	5.027×10^{-5}	1.062×10^{-4}	3.089×10^{-5}	7.945×10^{-5}	2.174×10^{-4}
	1.0	1.836×10^{-5}	5.408×10^{-5}	1.137×10^{-4}	3.035×10^{-5}	9.177×10^{-5}	2.428×10^{-4}
	2.0	1.935×10^{-5}	5.155×10^{-5}	1.443×10^{-4}	2.786×10^{-5}	9.714×10^{-5}	2.868×10^{-4}

3.6 Conclusion

We have defined a new resource allocation policy called Dynamic Aggregate Max-min Fairness (DAMF) in distributed job execution with task transmissions. We prove that DAMF satisfies several widely accepted properties, including Pareto efficiency, envy-freeness and strategy-proofness. We present an algorithm for computing the DAMF allocation and a fast approximation. Experimental evaluations using real job traces from Alibaba and Google demonstrate that DAMF could effectively enhance the fairness of resource allocation among jobs and improve the system utility. In the future, we plan to extend our work to deal with multiple resource types.

Chapter 4

Task Scheduling and Transmission for Distributed Job Execution

4.1 A Motivating Example

Fig. ?? provides a motivating example of the problem with four jobs J_1 , J_2 , J_3 and J_4 in three geo-distributed data centers (sites) S_1 , S_2 and S_3 . All jobs are submitted at time 0. For simplicity, we assume that each task of any job requires one computing slot and takes one time unit to process, and each site has one computing slot only. Fig. ?? shows the initial task distribution of each job among the three sites. The response time of a job is decided by the completion time of its last task across all sites. Obviously, different execution orders of the jobs can change the completion times of their tasks and result in different job response times. If the jobs are simply scheduled in the order of their indices (which, for example, represents their submission order), jobs J_1 , J_2 , J_3 and J_4 will complete at time 4, 8, 10 and 13, respectively, so the total job response time is $4 + 8 + 10 + 13 = 35$ time units. The optimal job execution order without task transmission is $\langle J_4, J_3, J_1, J_2 \rangle$ as shown in Fig. ??, and the resultant total job response time is $3 + 6 + 9 + 13 = 31$ time units, which is 4 time units better than the first schedule.

Suppose that with utilization of network resources, the input data of one task is allowed to be transferred between sites and it takes one time unit to arrive at the destination. Under the job execution order of Fig. ??, the optimal transmission can decrease the total job response time to 29 time units. For example, we can

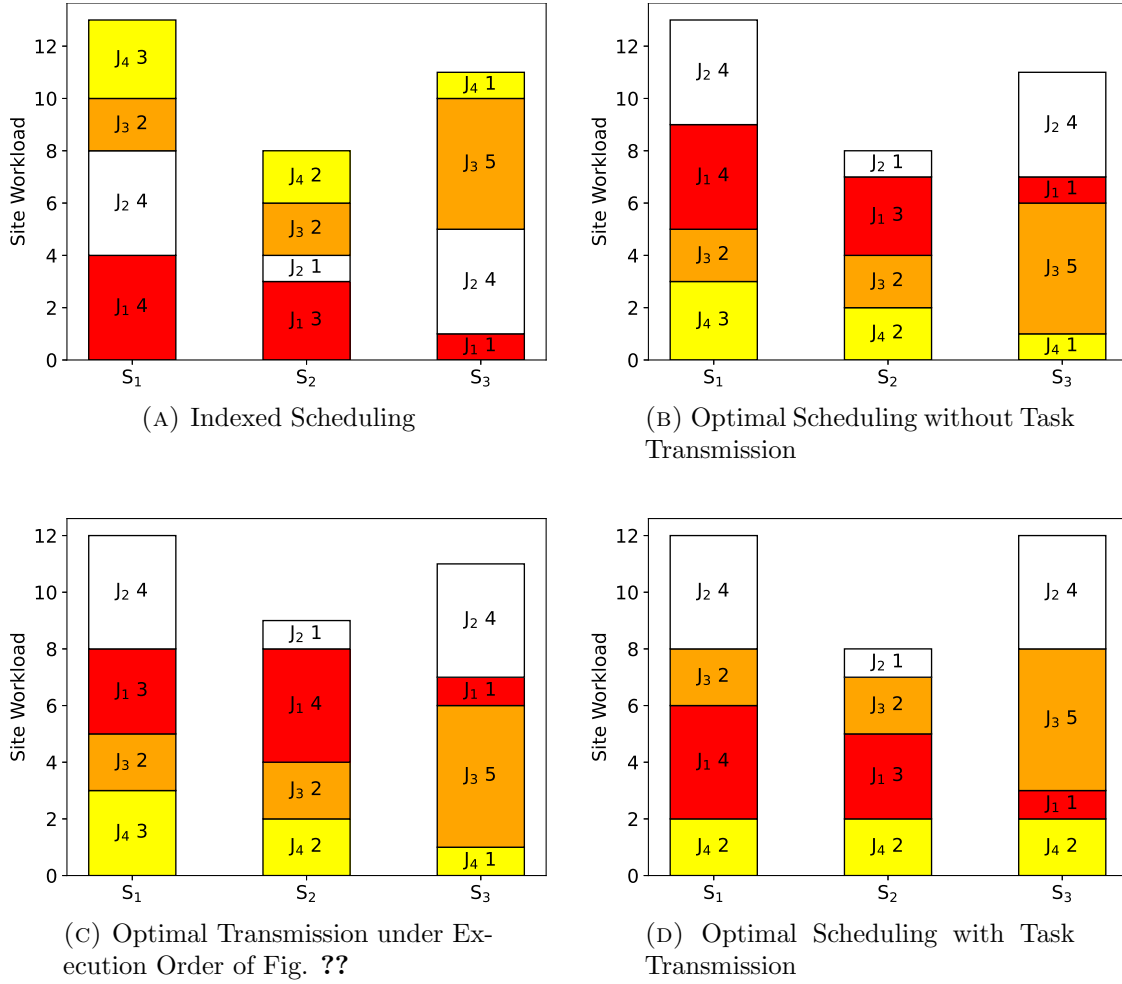


FIGURE 4.1: Motivating Example

transfer one task of J_1 from site S_1 to S_2 , as shown in Fig. ???. This task will arrive at site S_2 at time 1 when the slot of S_2 is processing a task of J_4 (and J_4 is scheduled earlier than J_1 in the execution order), so the task transferred can be treated as a local task of J_1 at site S_2 initially. Consequently, J_4 , J_3 , J_1 and J_2 will complete at time 3, 6, 8 and 12, respectively, and the total job response time is $3 + 6 + 8 + 12 = 29$ time units. In our example, the optimal solution is to transfer one task of J_4 from site S_1 to S_3 while modifying the job execution order to be $\langle J_4, J_1, J_3, J_2 \rangle$, as shown in Fig. ???. This results in the total job response time of $2 + 6 + 8 + 12 = 28$ time units, which is even better than the schedule in Fig. ??.

The above example illustrates some interesting phenomena. First, job execution order and task transmission have mutual influence. The optimal execution order of jobs can change after task transmissions are performed: both execution orders in Figs. ?? and ?? achieve the optimal total job response times given the respective

task transmissions, but J_1 and J_3 are swapped in the execution order after a task is transferred in Fig. ???. Vice versa, different job execution orders have different optimal task transmissions: the optimal transmission changes after we modify the job execution order from $\langle J_4, J_3, J_1, J_2 \rangle$ (Fig. ??) to $\langle J_4, J_1, J_3, J_2 \rangle$ (Fig. ??). Second, the transmission in the optimal solution is from site S_1 to S_3 , while the site with the lowest load is S_2 . This indicates that the transmissions that minimize the total job response time can differ from the one that focuses on load balancing, so traditional load balancing techniques may not be preferable in this efficiency-targeted scenario.

4.2 System Model

Consider a system consisting of m separate geo-distributed sites: $\mathcal{S} = \{S_1, S_2, \dots, S_m\}$. Each site S_j has compute resources with capacity u_j , which can be interpreted as the volume of computing slots available. A set of n jobs $J_1, J_2, \dots, J_n$ are submitted to run in the system. Each job J_i consists of tasks that require distinct data partitions and can run in parallel. For simplicity, we assume that each task takes one computing slot to process. In big data processing frameworks, tasks of the same job typically process data partitions of the same size [? ?], so they would take similar times to process. We denote the processing time of a task in each job J_i as l_i , which can be estimated by the historical execution records of the job or sampling-based learning, which proactively samples a small fraction of the tasks in the job to make an estimate [?]. A job is completed when all of its tasks are completed.

The data partitions required by the tasks of each job are originally distributed among the sites. Due to data locality consideration, each task is initially restricted to be executed at a certain site where its input data is available. Therefore, the computational demand of a job J_i on site S_j can be modeled as the number of its tasks that can run only on S_j initially. We represent the computational demands of all jobs across all sites as a matrix $D_{n \times m}$, where each element d_{ij} denotes job J_i 's computational demand on site S_j .

Suppose the sites in the system are connected by networks, which can be used to transfer data and hence tasks from one site to another. The network resources in

TABLE 4.1: Summary of Notations

Notation	Definition
$\mathcal{S} = \{S_1, S_2, \dots, S_m\}$	Set of sites
$U = \{u_1, u_2, \dots, u_m\}$	Compute resource capacities of individual sites
$\{J_1, J_2, \dots, J_n\}$	Set of jobs
$D_{n \times m}$	Compute resource demands
$B_{m \times m}$	Network resource availability
$\{l_1, l_2, \dots, l_n\}$	Task processing times of individual jobs
$\{r_1, r_2, \dots, r_n\}$	Data partition sizes of individual jobs

the system is denoted by a matrix $B_{m \times m}$, where each element b_{ij} represents the bandwidth available on the link from site S_i to site S_j . b_{ij} is set to 0 if there is no direct link between S_i and S_j . Let r_k denote the size of a data partition of job J_k , then a task of J_k takes r_k/b_{ij} time to transfer from site S_i to site S_j (if $b_{ij} = 0$, this time is defined to be infinite). For simplicity, we assume that the transmissions on each link are carried out sequentially. That is, if there is an ongoing transmission on the link, a new transmission has to wait and its arrival time at the destination site will be delayed.

We study the problem of deriving an execution order of jobs and a set of task transmissions between sites, so that the total job response (completion) time of all jobs is minimized. The key notations used in this paper are summarized in Table ???. For simplicity, we use $[x]$ to represent the set $\{1, 2, \dots, x\}$ for a positive integer x . Note that although the above model is formulated in a static setting, it can be easily extended to address online scenarios by dynamically capturing snapshots of job demand distributions and system resources.

4.3 Workload-Aware Selection with Transmission

4.3.1 Optimizing the Completion Time of a Single Job

We start by considering the subproblem of how to utilize network resources in the system to minimize the completion time of a single job. This subproblem can be formulated as follows: Given some workloads and task transmissions already scheduled in the system, and a job J_k with task processing time l_k , data partition

size r_k and initial demand distribution $d_{k1}, d_{k2}, \dots, d_{km}$ across the m sites, we would like to determine a set of task transmissions for J_k so that its job completion time can be minimized.

We use a sequence of arrays $W_1, W_2, \dots, W_m$ to represent the workload already scheduled for each computing slot at each site, where the workload of a slot refers to the total processing time of all tasks assigned to it (i.e., the time when this slot will become available). Each W_i is an array whose length is the number of computing slots at site S_i , and each element w_{ij} in W_i represents the scheduled workload of the j -th slot at S_i . In addition, we use a matrix $T_{m \times m}$ to represent the usage of network resources by the task transmissions already scheduled, where t_{ij} records the time when the link from site S_i to site S_j will complete the scheduled transmissions and become available.

To avoid the complex interaction between task transmissions and executions and simplify the problem model, we assume that every transferred task will arrive at its destination site before any computing slot at that site becomes available. This ensures that the transferred tasks can be processed continuously at their destination sites together with the tasks originally available there, which simplifies the modeling of job completion time. Based on this, we model the completion time of job J_k as follows. Suppose a matrix $X_{m \times m}$ represents the task transmissions of job J_k , where each element x_{ij} refers to how many tasks of J_k are to be transferred from site S_i to site S_j . Naturally, each x_{ij} is greater than or equal to 0. In particular, for each site S_i , we define $x_{ii} = 0$. We use an array Y of length m to represent the computational demands at the m sites after all the transmissions in X are performed. Each element y_i refers to the number of tasks to be executed at site S_i after the transmissions, and is given by:

$$\forall i \in [m], \quad y_i = d_{ki} + \sum_{j=1}^m x_{ji} - \sum_{j=1}^m x_{ij}. \quad (4.1)$$

Recall that we assume each task transmission to arrive at its destination site before any computing slot at that site becomes available. Thus, all the y_i tasks of job J_k to be executed at site S_i can be scheduled by iteratively assigning each task to the slot of the lowest workload in S_i . Let $w_{ij}^{(t)}$ denote the workload of the j -th slot in site S_i after t tasks of J_k are assigned to the slots at S_i . $w_{ij}^{(t)}$ can be computed

iteratively as follows:

$$\forall j \in [u_i], \quad w_{ij}^{(t)} = \begin{cases} w_{ij}, & t = 0 \\ w_{ij}^{(t-1)} + l_k, & j = \operatorname{argmin}_{p \in [u_i]} w_{ip}^{(t-1)}, \\ w_{ij}^{(t-1)}, & j \neq \operatorname{argmin}_{p \in [u_i]} w_{ip}^{(t-1)}, \end{cases}$$

where in each iteration, a task is assigned to the slot of the lowest workload ($\operatorname{argmin}_{p \in [u_i]} w_{ip}^{(t-1)}$) and its workload then increases by l_k , while other slots are kept unchanged. Ties are broken arbitrarily if multiple slots have the same lowest workload. Based on this, the time when all y_i tasks complete execution at site S_i can be estimated as a function $f(W_i, y_i)$:

$$f(W_i, y_i) = \begin{cases} 0, & y_i = 0, \\ \min_{j \in [u_i]} (w_{ij}^{(y_i-1)} + l_k), & y_i > 0, \end{cases}$$

which gives the completion time of the last task at S_i . Note that function f 's inputs and outputs cannot be captured by any closed-form expression. The objective of optimizing the job completion time of J_k (i.e., $\max_{i \in [m]} f(W_i, y_i)$ — the completion time of its last task across all sites) can be achieved by searching for the best task transmissions X between sites, which decide the computational demands Y at different sites and hence job J_k 's completion time.

It is essential to determine the bounds of X before starting the search. An excessively tight bound can limit the potential benefit of task transmissions in reducing the job completion time, whereas an excessively loose bound may give rise to task transmissions that fail to reach the destination sites in time, thereby introducing additional delays in the job completion time. Suppose x_{ij} tasks are to be transferred from site S_i to site S_j . Then, the completion time of the last task will be at least $t_{ij} + x_{ij} \cdot r_k / b_{ij} + l_k$ (when it is processed immediately after arrival), and we require this last task to finish before the job completion time:

$$\forall i \in [m], \quad j \in [m], \quad t_{ij} + \frac{x_{ij} \cdot r_k}{b_{ij}} + l_k \leq \max_{i \in [m]} f(W_i, y_i). \quad (4.2)$$

In addition, the number of tasks transferred out of a site cannot exceed the number of tasks it holds initially, which imposes another upper bound on $\sum_{j=1}^m x_{ij}$ at each

site S_i :

$$\forall i \in [m], \quad 0 \leq \sum_{j=1}^m x_{ij} \leq d_{ki}. \quad (4.3)$$

Constraints (??) and (??) collectively define the upper bounds of the elements in X . Therefore, we can define a program to search for task transmissions X under constraints (??), (??) and (??) to optimize the job completion time $\max_{i \in [m]} f(W_i, y_i)$. If multiple transmission matrices X 's produce the same minimum job completion time, we choose the X with the least total number of task transmissions. The rationale is to minimize the network resource usage and make more network resources available to other jobs.

4.3.2 Transformation into ILP

The program in Section ?? cannot be solved directly by popular optimization solvers owing to the discrete function f , which cannot be captured by any closed-form expression. In order to make the program solvable, we transform it into solving multiple Integer Linear Programs (ILP) that are supported by popular solvers. To facilitate presentation, we refer to the completion time of the last task at a site S_i as "the completion time at site S_i ". We compute and record the completion times $f(W_i, y_i)$ at each site S_i using the method introduced in Section ??, for increasing number of tasks y_i scheduled at site S_i starting from $y_i = 0$. The calculation ends at the lowest y_i value where the resulting $f(W_i, y_i)$ at site S_i exceeds the job completion time when all tasks are executed locally without transmission, i.e., $\max_{i \in [m]} f(W_i, d_{ki})$.

Note that these $f(W_i, y_i)$ values include all possible completion times at each site that J_k can produce, and they are non-decreasing with respect to y_i since adding a new task can never reduce the completion time at a site. With these $f(W_i, y_i)$ values, given any target job completion time z , the maximum number of tasks c_i that each site S_i can process (to keep its completion time within this target) can be derived as $c_i = \max\{p \mid f(W_i, p) \leq z\}$. Thus, the target z can be converted to an array of upper bounds $c_1, c_2, \dots, c_m$ for the elements $y_1, y_2, \dots, y_m$ in Y . Hence, the problem of determining "whether the job J_k can achieve a target job completion time z using task transmissions" can be transformed to determining whether task transmissions can be performed to bound

Algorithm 4: Optimizing the Completion Time of a Single Job

Input: A set of sites $\mathcal{S}$ with compute capacities U and network resources B ;
 a job J_k to be scheduled with task processing time l_k , data partition
 size r_k and computational demand distribution $d_{k1}, d_{k2}, \dots, d_{km}$;
 current workload $W_1, W_2, \dots, W_m$ and network usage T ;
Output: J_k 's optimal job completion time and task transmissions;

Compute and record $f(W_i, y_i)$'s for each site S_i with increasing y_i until
 $f(W_i, y_i) \geq \max_{i \in [m]} f(W_i, d_{ki})$, using the method in Section ??;
 Merge and sort all $f(W_i, y_i)$ values to form an array ψ ;
 $low = 1$; $high = \psi.size$; $opt = 0$;

while $low < high$ **do**

$middle = \lfloor (low + high)/2 \rfloor$;
 $c_i = \max\{p \mid f(W_i, p) \leq \psi[middle]\}$, for each $i \in [m]$;
 Minimize $\sum_{i=1}^m \sum_{j=1}^m x_{ij}$, subject to;
 $\forall i \in [m], j \in [m], t_{ij} + \frac{x_{ij} * r_k}{b_{ij}} + l_k \leq \psi[middle]$;;
 $\forall i \in [m], 0 \leq \sum_{j=1}^m x_{ij} \leq d_{ki}$;;
 $\forall i \in [m], y_i = d_{ki} + \sum_{j=1}^m x_{ji} - \sum_{j=1}^m x_{ij}$;;
 $\forall i \in [m], y_i \leq c_i$;;
 all x_{ij} 's and y_i 's are non-negative integers;;

if the above ILP is feasible and admits a solution **then**

$opt = \psi[middle]$;
 $X \leftarrow x_{ij}$'s in the solution of the above ILP;
 $high = middle - 1$;

else

$low = middle + 1$;

return opt, X ;

the elements in $y_1, y_2, \dots, y_m$ by $c_1, c_2, \dots, c_m$. The latter can be formulated as an ILP, because its constraints are linear and the elements in X and Y are integers. Furthermore, the minimum job completion time that J_k can achieve must be among all the $f(W_i, y_i)$ values. Therefore, we can sort all values $f(W_1, 0), f(W_1, 1), \dots, f(W_1, c_1), f(W_2, 0), f(W_2, 1), \dots, f(W_2, c_2), \dots, f(W_m, 0), f(W_m, 1), \dots, f(W_m, c_m)$ to form an array ψ . Then, the minimum job completion time of J_k can be found by a binary search in ψ and solving multiple ILPs.

Algorithm ?? presents the details of optimizing the job completion time of job J_k . $\psi.size$ represents the number of elements in the array ψ . The algorithm first computes and records the possible completion times $f(W_i, y_i)$ at each site S_i using the method introduced in Section ?. These values are then merged and sorted to

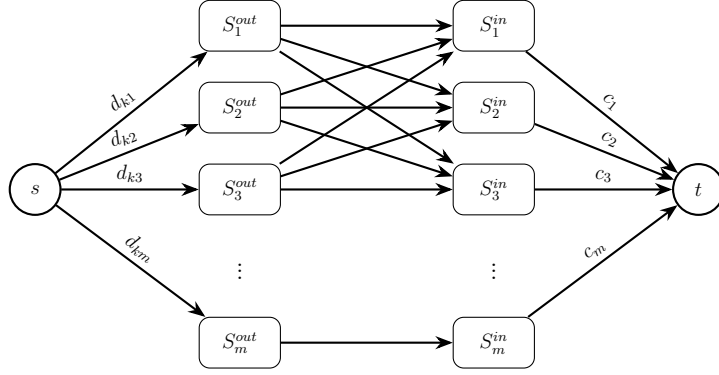


FIGURE 4.2: The flow network

form a new array ψ which records the job completion times that J_k can possibly achieve. Next, a binary search is applied to the array ψ to find the optimal job completion time in ψ as well as the transmissions required. The parameters *low* and *high* in the algorithm refer to the boundary indices of the binary search. In each iteration, the *middle* index and bounds $c_1, c_2, \dots, c_m$ are updated, and the feasibility of an ILP is checked to decide whether the job completion time $\psi[\textit{middle}]$ is achievable.

4.3.3 Transformation into Minimum-cost Flow Problem

Algorithm ?? solves the problem, but its computational complexity could raise concerns. ILP is NP-hard in general. If the solution space becomes huge (e.g., the job has thousands of tasks unevenly spread across tens of geo-distributed sites), solving the ILP can incur intolerable computational overhead. To improve the algorithm's efficiency, we further transform the ILP problem in lines 8-13 of Algorithm ?? to a minimum-cost flow problem in a bipartite flow network shown in Fig. ?. The flow network has a source node s , a sink node t , a set of m nodes on the left side and a set of m nodes on the right side. To facilitate exposition, we use S_i^{out} and S_i^{in} to represent the i -th node on the left and right sides, respectively. A set of m edges with cost 0 connects the source node s to the m nodes on the left, and the capacity of each edge (s, S_i^{out}) is set to d_{ki} . In addition, a set of $m \times m$ edges connects the m nodes on the left to the m nodes on the right. The capacity of each edge (S_i^{out}, S_j^{in}) where $i \neq j$ is set to the upper bound of x_{ij} given in line 9 of Algorithm ?? (where $\psi[\textit{middle}]$ is the target job completion time) and the cost of the edge is set to 1. The capacity of each edge connecting each pair of nodes with the same

index (S_i^{out}, S_i^{in}) is set to d_{ki} (the initial computational demand at site S_i) and the cost of the edge is set to 0. Finally, a set of m edges with cost 0 connect the m nodes on the right to the sink node t , and the capacity of each edge (S_i^{in}, t) is set to c_i (the upper bound of y_i for the target job completion time). The problem is to decide whether the bipartite flow network can transfer all $\sum_{i=1}^m d_{ki}$ units of flows from s to t under the given edge capacity constraints while minimizing the total cost.

Each edge (s, S_i^{out}) represents the initial number of tasks d_{ki} at site S_i . Each edge (S_i^{out}, S_j^{in}) where $i \neq j$ denotes the task transmissions from site S_i to site S_j . Due to the flow conservation constraint, the sum of flows going out of each node S_i^{out} is d_{ki} , ensuring that the number of task transmissions out of site S_i does not exceed d_{ki} . Each edge (S_i^{out}, S_i^{in}) denotes the number of local tasks that are not transferred out of site S_i , so it refers to no transmission and the cost is set to 0. Each edge (S_i^{in}, t) represents the final number of tasks to be executed at site S_i after transmissions, which is required to fall below the upper bound c_i . It is easy to verify that this minimum-cost flow problem is equivalent to the ILP problem in Section ???. Since all the edge capacities in the flow network are integers, based on the Integral Flow Theorem [?], the solution (if it exists) is made up of integral units of flows on all edges. Existing algorithms (e.g., Successive Shortest Path Algorithm, Cost Scaling Algorithm [?]) for solving the minimum-cost flow problem in bipartite graphs have polynomial time complexities. They can solve the problem more efficiently than ILP.

4.3.4 Optimizing the Completion Times of All Jobs

We have solved the problem of minimizing the completion time of a single job given the workloads and task transmissions already scheduled. We now consider how to deal with the problem when multiple jobs are submitted to run in the system. Since how a single job will utilize the network resources has been solved, the key problem that remains is how to decide an efficient job execution order. Inspired by the work from [?], we develop a Workload-Aware Selection with Transmission (WAST) algorithm, which aims to minimize the total job completion time by scheduling job execution and task transmission progressively across the system. In each iteration, WAST traverses all unscheduled jobs, searches for task

transmissions for each job to minimize its completion time (accounting for the workloads and task transmissions already scheduled), and selects the job with the earliest possible completion time to add to the execution order.

Algorithm 5: Workload-Aware Selection with Transmission

Input: A set of sites $\mathcal{S}$ with compute capacities U and network resources B ;
a set of jobs $J_1, J_2, \dots, J_n$ with task processing times $l_1, l_2, \dots, l_n$ and
data partition sizes $r_1, r_2, \dots, r_n$; a computational demand matrix D ;

Output: Job execution order and task transmissions;

$\mathcal{J} = \{J_1, J_2, \dots, J_n\}$;

Scheduled workload $W_i = \mathbf{0}$, for each $i \in [m]$;

Network usage $T = \mathbf{0}$;

while $\mathcal{J} \neq \emptyset$ **do**

for each job $J_k \in \mathcal{J}$ **do**

 Call Algorithm ?? on J_k given W and T to obtain J_k 's optimal
 completion time and the corresponding task transmissions X^k ;

 Find the job $J_{min} \in \mathcal{J}$ of the earliest completion time;

 Append J_{min} to the execution order and save its task transmissions X^{min} ;

$t_{ij} = t_{ij} + x_{ij}^{min} \cdot r_{min}/b_{ij}$, for each $i, j \in [m]$;

$y_i = d_{ki} + \sum_{j=1}^m x_{ji}^{min} - \sum_{j=1}^m x_{ij}^{min}$, for each $i \in [m]$;

for each $i \in [m]$ **do**

for each count $\in [y_i]$ **do**

$j = \operatorname{argmin}_{p \in [u_i]} w_{ip}$;

$w_{ij} = w_{ij} + l_{min}$;

$\mathcal{J} = \mathcal{J} \setminus \{J_{min}\}$;

return Job execution order and each job's task transmissions;

Algorithm ?? shows the details of WAST. Initially, there is no scheduled workload or task transmission, and the set of unscheduled jobs $\mathcal{J}$ contains all jobs (line 3). In each iteration, the algorithm computes the optimal completion time of each job in $\mathcal{J}$ using Algorithm ?? with ILP transformed into solving the minimum-cost flow problem (lines 5-6). Then, the job J_{min} with the earliest completion time is selected and added to the job execution order, and its task transmissions X^{min} are saved (lines 7-8). Meanwhile, the scheduled workload W and network usage T are updated based on those arising from J_{min} , and J_{min} is removed from $\mathcal{J}$ (lines 9-14). The algorithm terminates when all jobs are scheduled and returns the job execution order and task transmissions of each job.

Additional optimizations can be applied to enhance the algorithm's computational efficiency. We can reduce the number of binary search operations for each job by decreasing the number of elements to consider in the array ψ (the job completion times that the job can possibly achieve). First, duplicate elements can be removed from ψ . Next, suppose that infinite bandwidth is available on the links between all site pairs so that any task transmission is instant. In this case, the best strategy that minimizes job completion time is to treat all m sites as one aggregate site and iteratively assign each task to the slot with the lowest workload globally. The job completion time derived in this way is guaranteed to be no worse than the optimal solution in the case of finite bandwidth, so it is a lower bound on the optimal job completion time. As a result, all elements in ψ below this lower bound can be eliminated from consideration. Furthermore, in each iteration, suppose that job J_{min} has the minimum completion time t^* among all jobs examined before examining an unscheduled job J_i . If the above lower bound of J_i 's completion time is higher than or equal to t^* , there is no need to examine J_i in this iteration because it cannot achieve an earlier job completion time than t^* . Otherwise, if the lower bound of J_i 's completion time is lower than t^* , t^* can be set as an upper bound when searching in J_i 's array ψ of possible job completion times, since we are only concerned about whether J_i can achieve an earlier job completion time than t^* . In summary, when examining an unscheduled job J_i , we can remove all elements in ψ that fall below the lower bound of its completion time or exceed t^* .

4.3.5 Recursive Approximation

Algorithm ?? attempts to minimize the average job completion time in a step-by-step manner by solving multiple minimum-cost flow problems. We also present an alternative recursive approximation method called WAST_a which provides better computational efficiency. The outer loop of WAST_a is the same as WAST, and we only present the inner loop in Algorithm ?? which optimizes the completion time of a single job J_k . For simplicity, WAST_a assumes that, a site can either receive J_k 's tasks from other sites or send J_k 's tasks to other sites, but not both. Under this assumption, each task will be transferred at most once, because the destination site of any transmission will not send any task out. WAST_a first computes the completion time of J_k when all its tasks are executed locally without transmission as $q = \max_{i \in [m]} f(W_i, d_{ki})$ (line 8). Then, WAST_a starts a loop,

Algorithm 6: Recursive Approximation of Optimizing the Completion Time of a Single Job

Input: A set of sites $\mathcal{S}$ with compute capacities U and network resources B ;
 a job J_k to be scheduled with task processing time l_k , data partition size r_k and computational demand distribution $d_{k1}, d_{k2}, \dots, d_{km}$;
 current workload $W_1, W_2, \dots, W_m$ and network usage T ;

Output: J_k 's job completion time and transmissions;

Copy current workload: $W'_i = W_i$, for each $i \in [m]$;

$X = \mathbf{0}$;

$q = 0$;

for each $i \in [m]$ **do**

for each count $\in [d_{ki}]$ **do**

$j = \operatorname{argmin}_{p \in [u_i]} w_{ip}$;

$w_{ij} = w_{ij} + l_k$;

$q = \max\{q, w_{ij}\}$;

while true **do**

 Find a task whose completion time = q ;

 Suppose the task is currently assigned to the a -th slot of site S_g ;

if site S_g has ever received J_k 's tasks **then**

return q, X ;

$q^* = q, h = -1$;

for each $i \in [m]$ **and** $i \neq g$ **do**

if $t_{gi} + r_k/b_{gi} \leq \min_{p \in [u_i]} w_{ip}$ **and** $\min_{p \in [u_i]} w_{ip} + l_k < q^*$ **then**

$q^* = \min_{p \in [u_i]} w_{ip} + l_k$;

$h = i$;

if $h == -1$ **then**

return q, X ;

else

$x_{gh} = x_{gh} + 1$;

$t_{gh} = t_{gh} + r_k/b_{gh}$;

$w_{ga} = w_{ga} - l_k$;

$v = \operatorname{argmin}_{p \in [u_h]} w_{hp}$;

$w_{hv} = w_{hv} + l_k$;

$q = \max_{i \in [m]} f(W'_i, d_{ki} + \sum_{j=1}^m x_{ji} - \sum_{j=1}^m x_{ij})$;

where in each iteration it locates a task whose completion time is equal to q and tries to transfer it to another site to bring its completion earlier. The transmission is again required to arrive at the destination site before any computing slot at that site becomes available. In the case that multiple sites satisfy the requirement, the algorithm selects the one where the transferred task can be completed the earliest as the destination site (lines 14-18). After the task transmission is identified, the scheduled workload W , network usage T and J_k 's job completion time q and task transmissions X are updated accordingly (lines 22-27). Then, the algorithm moves onto the next iteration to locate another task whose completion time is equal to the new job completion time and look for its possible transmission. If the task whose completion time equals q is at a site that has received J_k 's tasks in previous iterations (line 12) or cannot find a destination site to have its completion time reduced (line 19), the algorithm terminates and returns all the transmissions identified in previous iterations and the job completion time q (lines 13 and 20). Note that this algorithm is not guaranteed to produce the optimal completion time for job J_k because it searches for task transmissions one at a time. However, it is easier to implement and takes much less time to compute the task transmissions.

4.3.6 Complexity Analysis

The outer loop of the WAST algorithm (Algorithm ??) has n iterations, in which it computes the optimal job completion times of all remaining jobs in $\mathcal{J}$ (which decreases by 1 in each iteration) using Algorithm ?. Therefore, Algorithm ? calls Algorithm ? for a total of $n(n+1)/2$ times. With ILP transformed into a minimum-cost flow problem, Algorithm ? solves multiple minimum-cost flow problems to derive a job's optimal completion time, and the number of problems to be solved depends on the job's task number, task distribution, data partition size as well as network resources available. Consider the worst case that a job J_k 's tasks concentrate at a single site initially and the network resources are adequate to ensure that these tasks can be transferred to any other site in time. In this case, all the tasks can be executed at any site, and the total number of possible job completion times for J_k is $F \cdot m$ (where F is the number of tasks in J_k), which requires $\log(F \cdot m)$ rounds in the binary search. The time complexity to solve the minimum-cost flow problem depends on the algorithm applied. Our flow network has $2m + 2$ nodes and $m^2 + 2m$ edges. The Successive Shortest Path Algorithm

(using a binary heap) has the worst-case computational complexity of $O(m^3 \log m)$ [?]. Therefore, assuming that all jobs have the same task number F , the worst-case computational complexity of Algorithm ?? is $O(n(n+1)/2 \cdot \log(F \cdot m) \cdot m^3 \log m) = O(n^2 m^3 \log F \log^2 m)$.

The outer loop of the approximation algorithm WAST_a is the same as that of Algorithm ?. In each iteration, WAST_a calls Algorithm ? to optimize the completion time of every remaining job in $\mathcal{J}$. Algorithm ? first places the tasks of the job locally, and then iteratively searches for destination sites to transfer tasks whose completion times are equal to the current job completion time. In the worst case, all tasks of the job are to be transferred to other sites to improve the job completion time (for example, all tasks of the job concentrate at one site initially whose scheduled workload is already the highest among all sites), and each task traverses all sites to find the best destination site, so the worst-case computational complexity is $O(F \cdot m)$. Therefore, if Algorithm ? calls Algorithm ? instead of Algorithm ?, the worst-case complexity is $O(n(n+1)/2 \cdot F \cdot m) = O(n^2 F m)$. Since no minimum-cost flow problem needs to be solved, this computational complexity is much lower than using Algorithm ?. We shall empirically evaluate the performance of these two algorithms in the experiments.

4.3.7 Deployment

In this section, we discuss how to apply the algorithms introduced above to online scenarios where jobs arrive dynamically for execution. The algorithm can be deployed at a central scheduler of the geo-distributed system. The scheduler keeps track of the system's resource capacities (including the compute resources and the network resources) as well as the demand distribution of each job across all sites. Once the job execution order and the task transmissions of each job are derived, the scheduler propagates them to relevant sites. Whenever a computing slot becomes available at a site S_i , S_i selects the first job in the job execution order that has pending tasks locally and assigns the slot to one task of this job. For each link from a source site to a destination site, the task input data of jobs are transferred based on the task transmission decisions. The transmissions follow the job execution order, so that the tasks of the jobs to be executed earlier are transferred earlier. The central scheduler recomputes the job execution order and the task transmissions

of jobs whenever a new job arrives. To do so, it takes a snapshot of the pending task distributions of all jobs as their computational demand distributions. The recomputation derives a new job execution order and new task transmissions for operation. Existing scheduled transmissions not yet completed are withdrawn, and the network usage T is reset to $\mathbf{0}$ for each recomputation.

4.4 Experiments

Simulation experiments using read-world job traces are conducted to evaluate the performance of applying our scheduling algorithms to online scenarios as discussed in Section ??.

4.4.1 Datasets

We use cluster job traces sourced from the Alibaba Cluster Trace Program [?], which contains cluster traces of different years from real production. We pre-process the raw trace files of years 2017 and 2018, extract the key information including the arrival time, task number, average memory consumption and task processing times of each job. A continuous sequence of 10,000 jobs is randomly selected from each trace to serve as the dataset driving our simulations. The job characteristics of selected sequences are shown in Fig. ?? and Fig. ?. Both sequences exhibit a heavy-tailed distribution, in which a small subset of jobs accounts for a disproportionately large share of the total computation load. We take the average memory

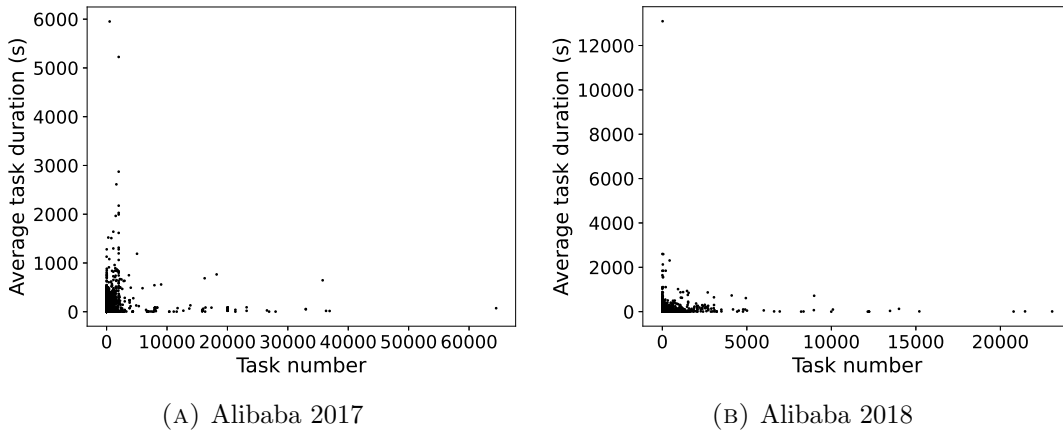


FIGURE 4.3: Job characteristic distribution of two datasets

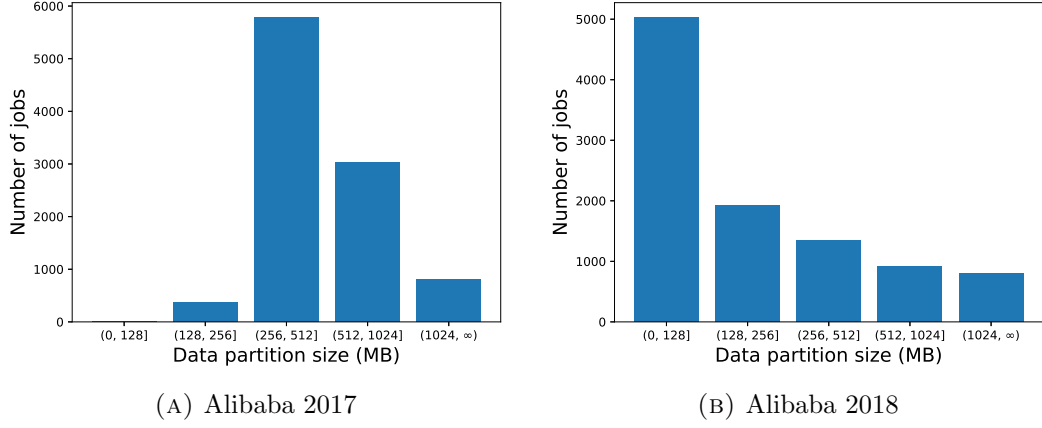


FIGURE 4.4: Data partition size distribution of two datasets

Characteristics	Alibaba 2017	Alibaba 2018
Average task number per job	266.01	94.70
Average task processing time (s)	129.14	82.41
Average data partition size (MB)	661.15	453.31

TABLE 4.2: Job characteristics of two datasets

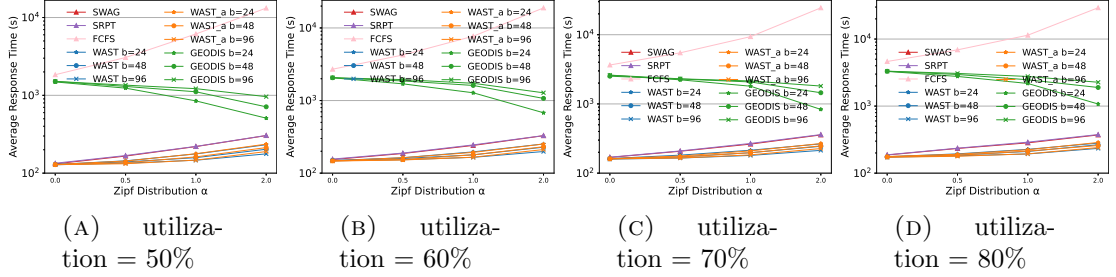


FIGURE 4.5: Average Job Response Time Compared with GEODIS, Alibaba 2018

consumption of each job as its data partition size. The average memory consumption records in the data traces are normalized values with respect to the memory capacity of the servers in the cluster. We derive the exact data partition size assuming that each server has 128GB RAM, under which the largest data partition is about 4GB. The average task number per job, average task processing time and average data partition size of all jobs are shown in Table ??.

4.4.2 System Configuration

We simulate a distributed system of m sites, each with the same number of computing slots. Each task is assumed to take up one computing slot to run and release it

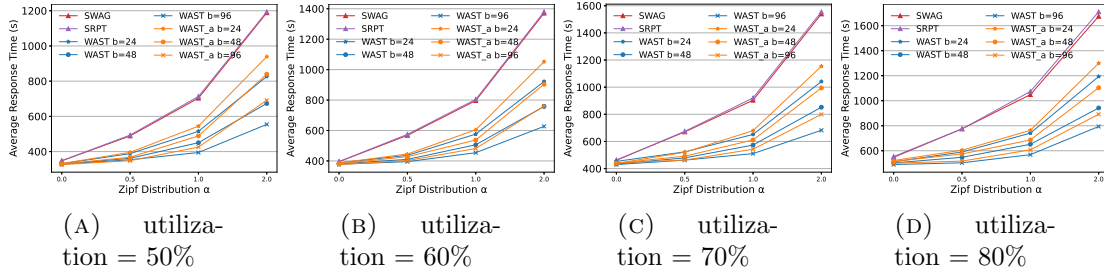


FIGURE 4.6: Average Job Response Time, Alibaba 2017

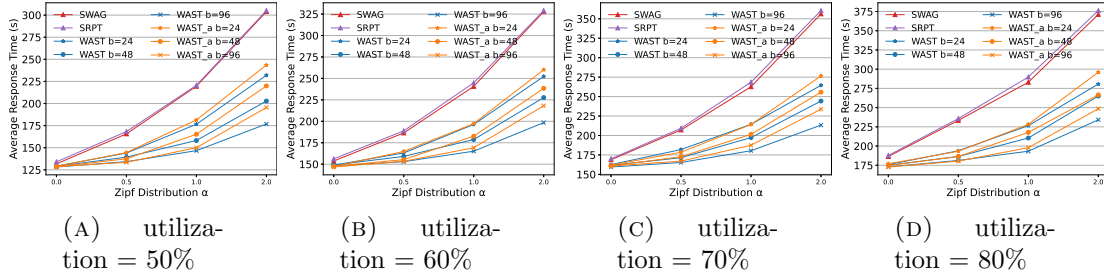


FIGURE 4.7: Average Job Response Time, Alibaba 2018

after completion. Due to the switching overhead consideration, we assume that the system does not allow preemption during task execution. For each job, the initial distribution of its tasks among the sites is generated by a Zipf distribution: the numbers of tasks placed at the m sites are proportional to $1, \frac{1}{2^\alpha}, \frac{1}{3^\alpha}, \dots, \frac{1}{m^\alpha}$, where α is a Zipf parameter. A higher α implies a more skewed distribution (a higher proportion of tasks are concentrated at fewer sites), whereas $\alpha = 0$ indicates a uniform distribution. In the experiments, we test four Zipf distributions with α set to 0, 0.5, 1 and 2 to simulate different task distributions of jobs. After the task distribution is generated, we shuffle the order of the sites when deploying each job's task distribution, so that the total workload at each site is roughly balanced. We scale the arrival times of the jobs in the datasets to simulate different levels of overall system utilization. We assume that the system is fully connected, i.e., a direct link exists between any two sites. The available bandwidth is set to 24Mbps, 48Mbps or 96Mbps for all links. We experiment with different numbers of sites, different numbers of computing slots per site, and different levels of system utilization. The results show similar performance trends. Due to space limitations, we focus on presenting the results for $m = 10$ sites, 32 computing slots per site, and system utilization = 50%, 60%, 70% and 80%.

4.4.3 Scheduling Policies

In Section ??, we have introduced the WAST algorithm and its fast approximation WAST_a. For comparison, we also simulate the following baseline algorithms.

- **First-Come-First-Serve (FCFS)**: FCFS simply schedules jobs in the order of their arrival times. When a computing slot becomes available, it is assigned to the job that arrives the earliest and has pending tasks.
- **Shortest Remaining Processing Time (SRPT)**: SRPT schedules jobs based on their estimated completion times in the case that each job runs alone in the system. More specifically, it estimates the completion time of a job J_k as $\max_{j \in [m]} \lceil d_{kj}/u_j \rceil \cdot l_k$, and executes pending jobs based on this estimation from low to high.
- **Workload-Aware Greedy Scheduling (SWAG)** [?]: SWAG greedily schedules jobs iteratively by estimating their completion times based on scheduled workloads without using network resources to transfer tasks between sites. Our WAST algorithm degenerates to SWAG when network resources are not available in the system.
- **GEODIS** [?]: GEODIS models job scheduling in geo-distributed data centers as a joint optimization of task placement and data access. It employs a heuristic scheduling algorithm which is triggered when any data center completes its scheduled workload. The objective of scheduling is to minimize the makespan of job execution (the latest job completion time) by selectively migrating data over faster network links when local servers are overloaded.

In the experiments, we use OR-Tools [?] to solve the minimum-cost flow problems in WAST. All algorithms (except FCFS) take the average task processing time of each job as input for scheduling and assume that the processing times of all tasks of the job are equal to the average. The simulation, on the other hand, uses the real processing times of individual tasks recorded in the trace to derive the job response time.

4.4.4 Experimental Results

Fig. ?? shows the performance of all algorithms in terms of average job response time. Due to space limitations, we only present the results for the Alibaba 2018 dataset (the results for the Alibaba 2017 dataset are similar). Note that the y-axis is plotted on a logarithmic scale. The results indicate that the average job response times of FCFS and GEODIS are one to two orders of magnitude higher than the other algorithms. This is because FCFS does not optimize job execution order and GEODIS is primarily designed to minimize the makespan.

Figs. ?? and ?? zoom into the performance of the remaining algorithms for both datasets. Compared to SRPT and SWAG, the average job response time of WAST and WAST_a is reduced by up to 53.3% for the Alibaba 2017 trace and 42.1% for the Alibaba 2018 trace. The improvement becomes more pronounced when the task distribution of jobs across sites is more skewed and when higher bandwidth is available, as more network resources can be effectively utilized to mitigate the imbalance in the task distribution of jobs under these conditions. Moreover, our WAST algorithm performs better for the Alibaba 2017 trace than the Alibaba 2018 trace. This is because the higher average task number per job in Alibaba 2017 leads to more tasks accumulated at some particular sites under highly uneven task distributions, resulting in increased response times if task transmission is not conducted. This, in turn, allows the WAST algorithm to offload more tasks from peak-load sites, enhancing the advantage of WAST. In addition, Figs. ?? and ?? show that although WAST_a compromises the quality of scheduling and performs slightly worse than WAST, it still significantly outperforms the baseline algorithms.

Figs. ?? and ?? show the 30th, 60th, 90th percentiles and the highest job response times for both datasets, under the Zipf distribution with $\alpha = 1$. For each percentile, the longest job response time among algorithms is normalized to 1, and the job response times of other algorithms are expressed relative to it. It can be seen that jobs in the lower percentiles have little improvement when the WAST and WAST_a algorithms are applied, since these jobs are small and have already achieved nearly best response times under the SRPT and SWAG strategies which favor small jobs. On the other hand, WAST and WAST_a can significantly enhance the performance of large jobs whose response times are relatively longer by transferring their tasks to mitigate their imbalanced task distributions. The highest job response time can be improved by more than 70%, as shown in Fig. ??.

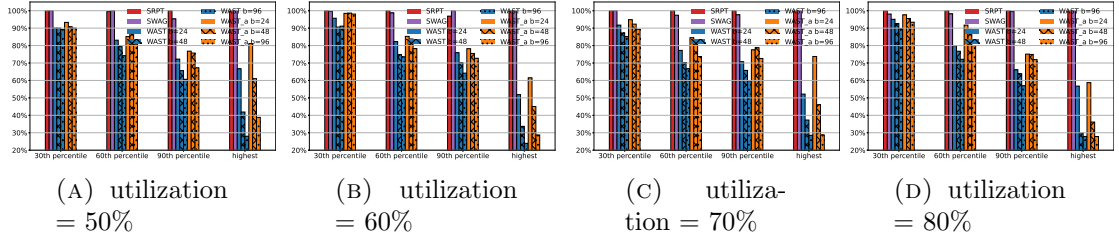


FIGURE 4.8: Job Response Time at Different Percentiles, Zipf Distribution $\alpha = 1$, Alibaba 2017

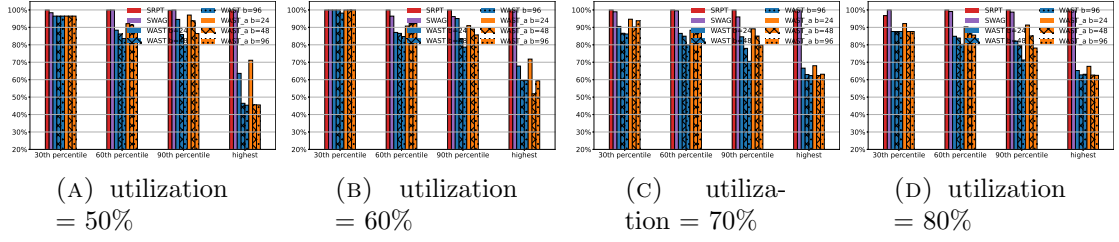


FIGURE 4.9: Job Response Time at Different Percentiles, Zipf Distribution $\alpha = 1$, Alibaba 2018

Tables ?? and ?? show the average computation time per recomputation of the SWAG, WAST and WAST_a algorithms under various bandwidth settings, slot numbers per site, Zipf parameters α and system utilization. Due to space limitations, we present the results for the Alibaba 2018 trace under 60% and 80% utilization only. Both tables demonstrate that WAST_a effectively reduces the computation time compared with WAST, which can be up to 87.3%. Furthermore, it can be seen that the average computation time of WAST is about an order of magnitude higher than that of SWAG, while the average computation time of WAST_a is of the same order of magnitude as SWAG, demonstrating the effectiveness of WAST_a in practice.

4.4.5 Evaluation on Robustness and Scalability

4.5 Discussion

Heterogeneous tasks. In real-world workloads, a job may consist of tasks with heterogeneous processing times and data sizes. Our proposed model and algorithms are capable of handling heterogeneous tasks by replacing the parameters l_k and d_k with task-specific values. Although this complicates the modeling of computation

TABLE 4.3: Average computation time (second), Alibaba 2018 trace, 60% utilization

slot	α	SWAG	b = 24Mbps		b = 48Mbps		b = 96Mbps	
			WAST	WAST _a	WAST	WAST _a	WAST	WAST _a
32	0.0	0.0157	0.1209	0.0201	0.1127	0.0199	0.1005	0.0198
	0.5	0.0184	0.1323	0.0323	0.1167	0.0297	0.1023	0.0274
	1.0	0.0253	0.1805	0.0453	0.1327	0.0468	0.1301	0.0424
	2.0	0.0475	0.3014	0.0756	0.2283	0.0872	0.1936	0.0877
64	0.0	0.0306	0.2293	0.0378	0.2310	0.0399	0.2283	0.0376
	0.5	0.0365	0.2797	0.0585	0.2467	0.0653	0.2330	0.0563
	1.0	0.0502	0.3646	0.0883	0.3281	0.0781	0.2867	0.0845
	2.0	0.0885	0.6217	0.1666	0.5365	0.1299	0.4369	0.1482
128	0.0	0.0772	0.6290	0.0899	0.6169	0.0928	0.6057	0.0922
	0.5	0.0876	0.7265	0.1127	0.7205	0.1228	0.6601	0.1341
	1.0	0.1302	0.9427	0.1459	0.9012	0.1565	0.8432	0.1781
	2.0	0.2068	1.5751	0.2247	1.4975	0.2317	1.3203	0.2493

TABLE 4.4: Average computation time (second), Alibaba 2018 trace, 80% utilization

slot	α	SWAG	b = 24Mbps		b = 48Mbps		b = 96Mbps	
			WAST	WAST _a	WAST	WAST _a	WAST	WAST _a
32	0.0	0.0326	0.2950	0.0411	0.2782	0.0398	0.2543	0.0393
	0.5	0.0402	0.3212	0.0785	0.2879	0.0616	0.2549	0.0547
	1.0	0.0521	0.4283	0.1078	0.3196	0.1100	0.2513	0.0905
	2.0	0.0964	0.4826	0.1503	0.5172	0.1785	0.4501	0.1746
64	0.0	0.0667	0.5337	0.0863	0.5363	0.0891	0.5340	0.0818
	0.5	0.0842	0.6638	0.1378	0.5871	0.1483	0.5706	0.1266
	1.0	0.1078	0.8532	0.2476	0.7262	0.1848	0.6623	0.2028
	2.0	0.1816	1.2361	0.3728	1.1003	0.2484	0.9437	0.2913
128	0.0	0.1665	1.3935	0.1963	1.4098	0.2024	1.4332	0.2087
	0.5	0.1947	1.7591	0.2664	1.6433	0.2899	1.6000	0.3193
	1.0	0.2715	2.1885	0.3210	2.0776	0.3473	1.8640	0.3722
	2.0	0.4260	3.9332	0.4998	3.6373	0.5136	3.0193	0.5459

and transmission process, it does not compromise the effectiveness of our proposed algorithms.

Limited network resources. Inter-datacenter bandwidth is often limited and costly compared to intra-datacenter links. Therefore, the scheduler must avoid excessive network resource consumption. We suggest that the algorithm should exclusively utilize the spare network bandwidth, which is identified based on current system conditions. The specific method for quantifying this spare bandwidth via network monitoring is beyond the scope of this paper.

Replicated data inputs. Cloud-based environments often replicate data chunks

across multiple geo-distributed servers in order to reduce service response latency and data transfer overhead [? ?]. Our method can be integrated with algorithms that determine task assignments under replicated data inputs. Specifically, we first execute the task assignment algorithm to determine an initial execution site for each task, and then employ our proposed algorithm(s) to derive the job execution order and task transmissions. Notably, if a destination site is found to already host a replica of the required data partition, the corresponding transmission is not needed, and the task can be processed locally at the destination.

4.6 Conclusion

In this paper, we have proposed a WAST algorithm and a fast approximation to improve job scheduling efficiency in geo-distributed systems with the usage of network resources. WAST constructs a near-optimal job execution order with task transmissions progressively. In each round, WAST searches for the job with the minimum completion time among all unsettled jobs given the scheduled workloads and task transmissions of previous rounds. The sub-problem of minimizing a single job's completion time is converted into solving multiple minimum-cost flow problems, which can be computed efficiently. We also present a fast approximation to WAST, which optimizes each job's completion time by transferring its tasks of the latest completion time to other sites recursively. We have evaluated WAST and its approximation using real job traces from Alibaba. The experimental results show that WAST can effectively reduce the average job response time, especially when the task distributions of jobs are highly skewed.

Chapter 5

Chapter5 Name

Quisque facilisis auctor sapien. Pellentesque gravida hendrerit lectus. Mauris rutrum sodales sapien. Fusce hendrerit sem vel lorem. Integer pellentesque massa vel augue. Integer elit tortor, feugiat quis, sagittis et, ornare non, lacus. Vestibulum posuere pellentesque eros. Quisque venenatis ipsum dictum nulla. Aliquam quis quam non metus eleifend interdum. Nam eget sapien ac mauris malesuada adipiscing. Etiam eleifend neque sed quam. Nulla facilisi. Proin a ligula. Sed id dui eu nibh egestas tincidunt. Suspendisse arcu.

5.1 Section 1

Quisque facilisis auctor sapien. Pellentesque gravida hendrerit lectus. Mauris rutrum sodales sapien. Fusce hendrerit sem vel lorem. Integer pellentesque massa vel augue. Integer elit tortor, feugiat quis, sagittis et, ornare non, lacus. Vestibulum posuere pellentesque eros. Quisque venenatis ipsum dictum nulla. Aliquam quis quam non metus eleifend interdum. Nam eget sapien ac mauris malesuada adipiscing. Etiam eleifend neque sed quam. Nulla facilisi. Proin a ligula. Sed id dui eu nibh egestas tincidunt. Suspendisse arcu.

5.2 Section 2

Quisque facilisis auctor sapien. Pellentesque gravida hendrerit lectus. Mauris rutrum sodales sapien. Fusce hendrerit sem vel lorem. Integer pellentesque massa vel augue. Integer elit tortor, feugiat quis, sagittis et, ornare non, lacus. Vestibulum posuere pellentesque eros. Quisque venenatis ipsum dictum nulla. Aliquam quis quam non metus eleifend interdum. Nam eget sapien ac mauris malesuada adipiscing. Etiam eleifend neque sed quam. Nulla facilisi. Proin a ligula. Sed id dui eu nibh egestas tincidunt. Suspendisse arcu.

Appendix A

Proofs for Part I or Chapter 3

A.1 Proof of Lemma

$$\psi^{av}(\theta) = \frac{1}{T} \int_0^T [\psi(\theta + \mu(\tau)) + C] \otimes \frac{\mu(\tau)}{a} d\tau$$

A.2 Proof of another Lemma

$$\begin{aligned} \gamma_1(\|x\|) &\leq W(t, x) \leq \gamma_2(\|x\|) \\ \frac{\partial W}{\partial t} + \frac{\partial W}{\partial x} \phi(t, x, 0) &\leq -\gamma_3(\|x\|) \end{aligned} \tag{A.1}$$

List of Author's Awards, Patents, and Publications¹

Awards

- Best Paper Awards, “A Great System,” *Nature*.

Patents

- A Great System, “A Great System,” *Nature*.

Journal Articles

- My name and My colleague, “A Great System,” *Nature*.

Conference Proceedings

- My name, My colleague 1, My colleague 3 and My colleague 3, “Greater System,” in *Conference of Vision, 2018*.

¹The superscript * indicates joint first authors

